

Felipe Carneiro Rodrigues
Matheus Junji Nakamura
Thiago Antici Rodrigues de Souza

**got.it: Sistema de gerenciamento de aprendizado
integrado com Agentes de Inteligência Artificial**

São Paulo, SP

2025

Felipe Carneiro Rodrigues
Matheus Junji Nakamura
Thiago Antici Rodrigues de Souza

got.it: Sistema de gerenciamento de aprendizado integrado com Agentes de Inteligência Artificial

Trabalho de conclusão de curso apresentado
ao Departamento de Engenharia de Computa-
ção e Sistemas Digitais da Escola Politécnica
da Universidade de São Paulo para obtenção
do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Fábio Levy Siqueira

São Paulo, SP

2025

Gerar a ficha catalográfica em <https://www.poli.usp.br/bibliotecas/servicos/catalogacao-na-publicacao>
Salvar o pdf e incluir na monografia

*Este trabalho é dedicado às pessoas que
ousaram sonhar e construir um mundo melhor.*

Agradecimentos

Ao Prof. Dr. Fábio Levy Siqueira, que nos ofereceu todo o suporte técnico e orientação durante o desenvolvimento deste trabalho. Agradecemos muito pela disposição, dedicação e conselhos neste período!

Aos nossos caros amigos do Centro Acadêmico e aos servidores da Universidade, que sempre sanaram as dúvidas ao longo da jornada acadêmica.

Às nossas famílias, que estão conosco desde o início de tudo, e nos incentivaram e apoiaram em todas as etapas que nos trouxeram até aqui. Palavras não são suficientes para expressar nossa imensa gratidão!

Resumo

Inspirado por aplicativos como Notion e Obsidian, este projeto de código aberto pretende unir as capacidades um sistema de anotações com um assistente de Inteligência Artificial, utilizando técnicas de *Retrieval-Augmented Generation* para fornecer preenchimento automático de frases, categorização dinâmica de notas, pesquisa semântica avançada e um sistema de perguntas e respostas contextualizado. O desenvolvimento envolve aplicar conceitos de Engenharia de *Software*, priorizando eficiência e usabilidade. A arquitetura será projetada para ser modular, escalável e *mobile-friendly*, valendo-se de ferramentas bem estabelecidas no mercado. Os principais desafios são a integração de todos os componentes da plataforma, sobretudo da *Large Language Model*, com os mecanismos de recuperação de contexto, otimização de desempenho e a implementação de um editor de texto com funcionalidades inovadoras.

Palavras-chave: Large Language Model. Inteligência Artificial. Retrieval-Augmented Generation. Aplicação educacional. Aplicação web. Engenharia de Software.

Abstract

Inspired by applications such as Notion and Obsidian, this open-source project aims to combine the capabilities of a note-taking system with an Artificial Intelligence assistant, using Retrieval-Augmented Generation techniques to provide automatic sentence completion, dynamic note categorization, advanced semantic search, and a contextualized question-and-answer system. The development involves applying Software Engineering concepts, prioritizing efficiency and usability. The architecture will be designed to be modular, scalable, and mobile-friendly, using well-established tools in the market. The main challenges are the integration of all the components of the platform, especially the Large Language Model, with context retrieval mechanisms, performance optimization, and the implementation of a text editor with innovative features.

Keywords: Large Language Model. Artificial Intelligence. Retrieval-Augmented Generation. Educational application. Web application. Software engineering.

Lista de ilustrações

Figura 1 – Sugestão de agenda para a <i>Lean Inception</i>	18
Figura 2 – Descrição da jornada da prática de exercícios na revisão com o auxílio de IA.	29
Figura 3 – Funcionalidades agrupadas e classificadas, resultado das etapas 3.1.5, 3.1.6 e 3.1.7.	30
Figura 4 – Canvas do MVP.	32
Figura 5 – Cronograma do projeto em diagrama de Gantt.	32
Figura 6 – Ilustração da aplicação <i>web</i> do <i>got.it</i> , mostrando as camadas principais e seus componentes.	40
Figura 7 – Fluxo de trabalho do Git adotado no projeto, ilustrando a estratégia de versionamento e CI/CD.	41
Figura 8 – Arquitetura básica do Kubernetes	42
Figura 9 – Arquitetura de rede do <i>cluster</i>	43
Figura 10 – Arquitetura de persistência de dados	44
Figura 11 – Fluxo GitOps com ArgoCD	45
Figura 12 – Estrutura de deploy das aplicações	46
Figura 13 – Diagrama de entidade-relacionamento do <i>got.it</i>	50
Figura 14 – Resultados do Maze para a tarefa de "Criar um subtítulo H2" com monitoramento de <i>misclicks</i> e tempo médio em cada tela	60
Figura 15 – Resultados do Maze para a tarefa de "Duplicar uma nota" com monitoramento de <i>misclicks</i> e tempo médio em cada tela	60
Figura 16 – Gráfico de WAU (<i>Weekly Active Users</i>) e DAU (<i>Daily Active Users</i>) no <i>got.it</i>	62
Figura 17 – Visão geral do produto final com destaque para o editor de notas. . . .	64
Figura 18 – Descrição da persona do estudante de economia.	70
Figura 19 – Descrição da persona do estudante de computação.	71
Figura 20 – Descrição da persona do estudante de medicina.	72
Figura 21 – Descrição da jornada do processo de aprendizado exploratório.	73
Figura 22 – Descrição da jornada de revisão.	74

Lista de abreviaturas e siglas

ACID	<i>Atomicity, Consistency, Isolation, Durability</i> (Atomicidade, Consistência, Isolamento e Durabilidade)
AGI	<i>Artificial General Intelligence</i> (Inteligência Artificial Geral)
AI/IA	<i>Artificial Intelligence</i> (Inteligência Artificial)
API	<i>Application Programming Interface</i>
CRD	<i>Custom Resource Definition</i>
CRUD	<i>Create, Read, Update, Delete</i>
CSS	<i>Cascading Style Sheets</i>
GPT	<i>Generative Pre-trained Transformer</i>
GPU	<i>Graphics Processing Unit</i>
HA	<i>High Availability</i> (Alta Disponibilidade)
IDE	<i>Integrated Development Environment</i>
JSON	<i>JavaScript Object Notation</i>
K8s	Kubernetes (abreviação)
LLM	<i>Large Language Model</i>
LoRA	<i>Low-Rank Adaptation</i>
MVP	<i>Minimum Viable Product</i> (Produto Mínimo Viável)
NoSQL	<i>Not Only SQL</i>
PEFT	<i>Parameter-Efficient Fine-Tuning</i>
RAG	<i>Retrieval-Augmented Generation</i>
SQL	<i>Structured Query Language</i>
UI	<i>User Interface</i> (Interface do Usuário)
UX	<i>User Experience</i> (Experiência do Usuário)
VRAM	<i>Video Random Access Memory</i>

Sumário

1	INTRODUÇÃO	12
1.1	Objetivos	12
1.2	Justificativa	12
1.3	Trabalhos Relacionados	13
1.3.1	Assistentes de Código com IA	13
1.3.2	Aplicativos de Flashcards	13
1.3.3	Aplicativos de Anotações	13
1.3.4	Comparativo	14
1.4	Organização do Trabalho	14
2	ASPECTOS CONCEITUAIS	16
2.1	<i>Lean Inception</i>	16
2.2	<i>Scrum</i>	18
2.3	LLMs (Large Language Models)	19
2.4	Técnicas para construção de aplicações com LLMs	20
2.4.1	<i>Retrieval-Augmented Generation (RAG)</i>	20
2.4.1.1	Funcionamento do RAG	21
2.4.2	Tool-Use	21
2.4.3	Fine-Tuning	22
2.4.3.1	Métodos de Fine-Tuning	22
2.5	Aspectos de Aplicação	23
2.5.1	Aspectos de Backend	23
2.5.2	Aspectos de Interface	23
2.6	Aspectos de Infraestrutura	24
2.6.1	Orquestração e Implantação	24
2.6.2	Armazenamento e Persistência	25
2.6.3	Rede e Hospedagem	25
2.6.4	Monitoramento	25
2.6.5	Versionamento de Código	26
3	MÉTODO DO TRABALHO	27
3.1	Levantamento de requisitos via <i>Lean Inception</i>	27
3.1.1	Visão do Produto	27
3.1.2	É - Não é - Faz - Não faz	27
3.1.3	Descrição das personas	28
3.1.4	Jornadas dos Usuários	29

3.1.5	Brainstorming de funcionalidades	29
3.1.6	Revisão Técnica, de UX e de Negócios das Funcionalidades	29
3.1.7	Sequenciador	30
3.1.8	Canvas MVP	31
3.2	Processo de desenvolvimento	32
4	ESPECIFICAÇÃO DE REQUISITOS	33
4.1	MVP: Tomada de anotações e interação com o modelo	33
4.2	Incremento 1: Recursos adicionais para as notas	34
4.3	Incremento 2: Recursos adicionais para a aplicação	35
4.4	Incremento 3: Recursos avançados das notas	35
4.5	Incremento 4: Recursos de priorização e planejamento de estudos	36
4.6	Requisitos Não Funcionais	37
4.7	Requisitos de Infraestrutura e Operações	37
5	DESENVOLVIMENTO DO TRABALHO	39
5.1	Projeto e Implementação	39
5.1.1	Visão Geral da Arquitetura	39
5.1.1.1	Camada de Aplicação	39
5.1.1.2	Camada de Infraestrutura	41
5.1.1.2.1	Conceitos Básicos do Kubernetes	42
5.1.1.2.2	Redes	42
5.1.1.2.3	Persistência de Dados	43
5.1.1.2.4	Aplicações e Entrega Contínua	45
5.1.1.2.5	Monitoramento e Observabilidade	46
5.1.1.2.6	<i>Deploy</i> das Aplicações	46
5.1.1.3	Fluxo de Dados	47
5.1.1.4	Decisões Arquiteturais Principais	48
5.1.2	Modelagem de Dados e Schema do Banco	49
5.1.2.1	Design Polimórfico com União Discriminada	50
5.1.2.2	Separação entre <i>Files</i> e <i>FileContents</i>	51
5.1.2.3	Estados Transitórios e <i>Optimistic UI</i>	52
5.1.2.4	Indexação Multi-modal para Busca	52
5.1.2.5	Armazenamento de <i>Embeddings</i> : Co-localização versus Tabela Separada	53
5.1.2.6	Segurança <i>Multi-tenant</i> e Isolamento de Dados	53
5.1.2.7	<i>Soft Delete</i> para Recuperação de Dados	53
5.1.3	Implementação do Sistema de Inteligência Artificial	54
5.1.3.1	Treinamento e Adaptação do Modelo Qwen	54
5.1.4	Desenvolvimento do <i>Backend</i>	56
5.1.4.1	Organização e Estrutura de Módulos	56

5.1.4.1.1	Organização Interna dos Módulos	56
5.1.4.2	Padrões de Desenvolvimento e Fluxos	57
5.1.5	Desenvolvimento do <i>Frontend</i>	57
5.1.5.1	Gerenciamento de Estado	57
5.1.5.2	Editor de Texto Rico	58
5.1.5.3	Integração com Backend	59
5.2	Testes e Avaliação	59
5.2.1	Validação dos Requisitos Mapeados no <i>Lean Inception</i>	59
5.2.2	Testes de Interface Visual com o Maze	59
5.2.3	Pesquisa de Funcionalidades Desejadas com Google Forms	61
5.2.4	Análise de Métricas de Uso com PostHog e Grafana	62
5.2.5	Testes automatizados	63
6	CONSIDERAÇÕES FINAIS	64
6.1	Resultados e Conclusões	65
6.2	Contribuições	65
6.3	Perspectivas de Continuidade	66
	 REFERÊNCIAS	 68
	 APÊNDICES	 69
	APÊNDICE A – DESCRIÇÕES DAS PERSONAS	70
	APÊNDICE B – JORNADAS DO USUÁRIO	73

1 Introdução

O final de 2022 marcou o início de uma nova era para a inteligência artificial, impulsionada pela popularização das LLMs (*Large Language Models*), especialmente com o lançamento do ChatGPT pela empresa americana OpenAI. Essa ferramenta generativa responde a solicitações em linguagem natural (*prompts*), utilizando padrões aprendidos a partir de uma grande base de dados de treinamento, composta por centenas de bilhões de parâmetros – como ocorre no modelo GPT-4, por exemplo. Nesse contexto, inúmeras soluções tecnológicas baseadas em LLMs têm sido desenvolvidas, buscando agilizar a produção de conteúdo, reduzir erros e automatizar tarefas repetitivas. No entanto, o potencial dessas ferramentas ainda não foi plenamente explorado, especialmente no campo educacional. Assim, uma plataforma de estudos de código aberto (*open-source*), integrada com modelos de inteligência artificial, pode contribuir para o aprendizado dos estudantes, oferecendo suporte personalizado e eficiente.

1.1 Objetivos

Este trabalho pretende desenvolver uma plataforma *web* educacional, focada na experiência do usuário, que permita criar anotações interativas, que facilitem o aprendizado e a busca de conhecimento. Essa ferramenta permite a organização e recuperação de notas, além de facilitar o fluxo de estudo com sugestões contextuais e geração de conteúdo automatizado, usando modelos de LLM *open-source*.

1.2 Justificativa

Em um contexto marcado pela preocupação no meio educacional e acadêmico com relação ao plágio e outros prejuízos ao aprendizado ocasionado pelo abuso das LLMs por parte dos discentes, este trabalho demonstra sua utilidade como ferramenta auxiliadora, reduzindo o tempo empregado pelos estudantes em tarefas repetitivas como geração de resumos e organização das anotações. Além disso, tendo em vista os altos custos envolvidos no uso intensivo de uma ferramenta de inteligência artificial, como observado nas soluções de mercado atualmente, o projeto traz como diferencial a adoção de um modelo *open-source* e *open-weights* (modelos com parâmetros abertos e sem restrição de usos) e o uso de *frameworks* e ferramentas de código aberto, reduzindo os custos dos usuários.

1.3 Trabalhos Relacionados

Nesta seção, são apresentadas soluções existentes no mercado que abordam problemas similares ao proposto neste trabalho. A análise dessas ferramentas permite identificar lacunas e oportunidades para o desenvolvimento do sistema.

1.3.1 Assistentes de Código com IA

O GitHub Copilot é um assistente de programação baseado em modelos de linguagem de grande escala, como o Codex e o GPT-4, desenvolvido pela GitHub em parceria com a OpenAI. A ferramenta interpreta comandos em linguagem natural e fornece sugestões automáticas de código diretamente no editor de texto, integrando-se a ambientes de desenvolvimento como Visual Studio Code e JetBrains.

Entre as vantagens do GitHub Copilot, destacam-se a integração nativa com IDEs populares e a capacidade de fornecer sugestões contextuais baseadas no código já escrito pelo desenvolvedor. No entanto, a ferramenta apresenta limitações relevantes para o contexto educacional: seu foco está exclusivamente no desenvolvimento de software, não oferecendo suporte para anotações de estudo ou organização de conhecimento. Além disso, exige uma assinatura paga, individual ou corporativa, e seu código-fonte é proprietário.

1.3.2 Aplicativos de Flashcards

O Anki é um aplicativo de código aberto para geração e uso de cartões de memória virtuais (*flashcards*). É frequentemente utilizado por estudantes de idiomas, Ciências da Saúde, Ciências Humanas e demais áreas de conhecimento que exigem memorização intensiva de conceitos. O aplicativo permite a inserção de imagens, símbolos especiais e diagramas, além de implementar algoritmos de repetição espaçada para agendamento automático de revisões, reforçando o aprendizado de tópicos mais difíceis.

Apesar de suas qualidades, o Anki apresenta desvantagens significativas. A curva de aprendizado na criação de cartões é íngreme, desencorajando iniciantes e prejudicando o processo de aprendizado. O aplicativo requer a importação de um “baralho” pronto ou o esforço do usuário em criar um manualmente. Os baralhos prontos de determinados assuntos, por sua vez, normalmente exigem pagamento. A ausência de integração com inteligência artificial generativa para criação automática de cartões representa uma lacuna importante.

1.3.3 Aplicativos de Anotações

O Notion é um *workspace* colaborativo que combina funcionalidades de notas, banco de dados e gerenciamento de projetos. A ferramenta oferece grande flexibilidade por

meio de templates personalizáveis e permite colaboração em tempo real entre usuários. Entretanto, as funcionalidades de inteligência artificial integradas ao produto exigem um plano pago. Além disso, os dados são armazenados em nuvem proprietária e o código-fonte não é aberto.

O Obsidian é um editor de notas em formato *Markdown* que se destaca pelo suporte a *links* bidirecionais e pela visualização do grafo de conhecimento. O aplicativo armazena os dados localmente e possui um ecossistema extensível por meio de *plugins*. Contudo, as funcionalidades de IA são disponibilizadas apenas por meio de *plugins* pagos de terceiros, e a curva de aprendizado para configuração avançada pode ser elevada.

Em ambos os casos, o uso da inteligência artificial impõe desafios relacionados às limitações dos modelos: a ausência de transparência na geração de raciocínio e a possibilidade de entrega de conteúdo incorreto devido a restrições dos dados de treinamento ou dos algoritmos, fenômeno conhecido como "alucinações".

1.3.4 Comparativo

A análise das soluções existentes revela lacunas que o presente trabalho busca preencher. Ferramentas como o GitHub Copilot focam exclusivamente em desenvolvimento de *software*, enquanto aplicativos de *flashcards* como o Anki exigem criação manual de conteúdo. Aplicativos de anotações como Notion e Obsidian oferecem inteligência artificial apenas em planos pagos e com código proprietário. O sistema proposto neste trabalho diferencia-se por:

1. Focar especificamente no contexto educacional de estudos e anotações;
2. Integrar geração automática de *flashcards* via IA;
3. Adotar modelos *open-source* e *open-weights*, reduzindo custos;
4. Estudar técnicas como RAG (*Retrieval-Augmented Generation*) para mitigar limitações dos modelos de linguagem.

1.4 Organização do Trabalho

Os demais capítulos do trabalho estão descritos a seguir.

- O Capítulo 2 apresenta os aspectos conceituais do trabalho, incluindo a metodologia *Lean Inception*, o *framework Scrum* para gestão do desenvolvimento, os fundamentos de *Large Language Models* (LLMs) e técnicas como RAG (*Retrieval-Augmented Generation*), *Tool-Use* e *Fine-Tuning*. Além disso, são descritos os aspectos de aplicação e infraestrutura utilizados no projeto.

- O Capítulo 3 expõe o processo detalhado de levantamento de requisitos, público-alvo e métricas de qualidade, de acordo com as recomendações do *Lean Inception*.
- O Capítulo 4 detalha os requisitos do projeto, descritos em função de "histórias de usuário", divididos em tarefas e subtarefas, critérios de aceite, e *personas* contempladas, com base no trabalho desenvolvido nos capítulos anteriores.
- O Capítulo 5 exhibe as decisões tomadas para a implementação efetiva do projeto.
- Finalmente o Capítulo 6 apresenta as considerações finais sobre o desenvolvimento do projeto.

2 Aspectos Conceituais

Este capítulo apresenta os fundamentos conceituais que embasam o desenvolvimento do projeto. Inicialmente, são descritas as metodologias *Lean Inception* e *Scrum*, utilizadas para o levantamento de requisitos e gestão do desenvolvimento, respectivamente. Em seguida, são apresentados os conceitos de *Large Language Models* (LLMs), incluindo a arquitetura de Transformadores e o mecanismo de atenção. São também detalhadas as técnicas para construção de aplicações com LLMs, como *Retrieval-Augmented Generation* (RAG), *Tool-Use* e *Fine-Tuning*. Por fim, são descritos os aspectos de aplicação e infraestrutura adotados no projeto.

2.1 *Lean Inception*

O *Lean Inception* é uma metodologia colaborativa criada por Paulo Caroli para alinhar equipes e *stakeholders* no início de projetos ágeis. O objetivo é definir, de forma rápida e eficiente, o Produto Mínimo Viável (MVP) a ser desenvolvido. A metodologia combina práticas do *Design Thinking* e do *Lean Startup* em uma série de atividades estruturadas, geralmente realizadas ao longo de uma semana, que permitem à equipe compreender o problema, definir a solução e priorizar as funcionalidades essenciais para a primeira versão do produto.

Abaixo estão descritas as etapas de elaboração do *Lean Inception*, conforme definidas em (CAROLI, 2018).

1. **Kick-off**: Apresentação do projeto, dos participantes e dos objetivos da semana. Criação de um ambiente colaborativo para as próximas etapas.
2. **Visão do produto**: Fornece uma descrição breve para guiar qual será a característica principal do produto, e qual será o posicionamento da equipe. Conforme descrito em (CAROLI, 2018), a visão é uma frase que segue o seguinte modelo:
Para: [cliente final], cujo: [problema que precisa ser resolvido]. O: [nome do produto] é um: [categoria do produto] que: [benefício-chave, razão para adquiri-lo], diferentemente do: [alternativa da concorrência]. O nosso produto: [diferença-chave].
3. **É - Não é - Faz - Não faz**: Detalha-se mais profundamente o que o produto é, e as suas principais funcionalidades, ajuda também no processo de limitação de escopo, destacando justamente o que o produto não é, e não faz.
4. **Descrição das personas**: Apresenta o perfil, comportamento e necessidades dos usuários finais que compõem o público alvo, considerando suas dores, objetivos e

contexto. Esse processo ajuda na concretização de como é o dia a dia de uma pessoa que usa o produto, e quais são as expectativas que a mesma pode ter ao usá-lo.

5. **Jornadas dos Usuários:** Mapeamento dos caminhos que os usuários percorrem ao interagir com o produto. Através dessa análise, é possível entender os pontos de contato, identificar possíveis fricções e otimizar a experiência do usuário. Além disso, a jornada ajuda a visualizar o fluxo de uso do produto desde o primeiro contato até a realização de seus objetivos.
6. ***Brainstorming* de funcionalidades:** Dinâmica criativa para gerar ideias sobre possíveis funcionalidades do produto. Neste momento, o objetivo é explorar livremente diferentes possibilidades, sem julgamentos ou limitações, para garantir uma ampla gama de ideias antes de aplicar filtros técnicos e estratégicos.
7. **Revisão Técnica, de UX e de Negócios das Funcionalidades:** Avaliação crítica das funcionalidades sugeridas durante o brainstorm. O foco aqui é analisar a viabilidade técnica, o impacto na experiência do usuário e o valor de negócio que cada funcionalidade pode gerar. Essa etapa serve para filtrar ideias e focar em soluções que sejam tanto eficientes quanto realizáveis.
8. **Sequenciador de Funcionalidades:** Organização e priorização das funcionalidades definidas, levando em consideração o valor para o usuário, a complexidade de implementação e a relevância para o MVP. O sequenciador de funcionalidades ajuda a estruturar um plano de execução, definindo a ordem lógica de desenvolvimento.
9. **Canvas MVP:** Consolidação de todas as informações levantadas durante a *Lean Inception* em um quadro visual que define o escopo do MVP. Esse canvas apresenta as principais funcionalidades, o público-alvo, os objetivos do produto e as métricas de sucesso, servindo como um guia claro para o time de desenvolvimento.
10. ***Showcase da Lean Inception:*** Apresentação dos resultados obtidos ao longo da semana para os *stakeholders*. Esse momento é dedicado a validar o escopo do MVP, revisar os principais aprendizados e definir os próximos passos. O *showcase* garante o alinhamento final entre todas as partes envolvidas antes do início do desenvolvimento.

Figura 1 – Sugestão de agenda para a *Lean Inception*.

Fonte: (CAROLI, 2018)

2.2 Scrum

O *Scrum* foi a metodologia adotada para conduzir efetivamente o desenvolvimento do projeto. As principais vantagens que levaram ao seu uso, conforme descritas nas seções de resumo de (SUTHERLAND, 2016), estão abaixo:

- **"Mude ou morra":** Ao longo de um projeto de desenvolvimento de *software*, é necessário que os desenvolvedores se adaptem e verifiquem a possibilidade de melhorias e modificações no projeto já desenvolvido.
- **"Fracasse rápido para que possa corrigir o problema o quanto antes":** O desenvolvimento incremental aliado a avaliações rápidas de valor de cada um dos ciclos de desenvolvimento do projeto permite eliminar tarefas que constituem um desperdício de tempo ou corrigir erros de funcionalidade em algo que foi recém desenvolvido.
- **"O tempo é finito. Trate-o como tal":** A divisão do trabalho em intervalos de tempo curtos, regulares e definidos (*sprints*) auxilia a produção de novas funcionalidades ao fim de cada um deles, que podem ser demonstradas e testadas.
- **"Torne o trabalho visível":** O *Scrum* propõe o uso de um quadro que consolida as tarefas e seu estado atual (a fazer, em andamento e realizado), facilitando o fluxo de atribuição de tarefas.

Dessa forma, a equipe pôde, com o auxílio do *framework*, transpor as tarefas obtidas com o *Lean Inception* para compor o *backlog* do produto. Em uma reunião futura, serão agendadas as *Sprint Plannings* (reuniões antes do início de cada *sprint* para alinhamento da equipe das tarefas a serem realizadas), as *Daily Scrums* (encontros breves, tipicamente

de menos de 15 minutos, nos quais os integrantes atualizam os status de cada tarefa), e as *Sprint Reviews* (em que os membros da equipe apresentam ao orientador todas as funcionalidades que atendem à definição de pronto especificada após o final de cada *sprint*).

2.3 LLMs (Large Language Models)

A Inteligência Artificial passou por diversas transformações ao longo dos anos, com períodos de intenso otimismo seguidos por fases de retração. Na década de 1950, o campo despontou com ideias e experimentos pioneiros, gerando grandes expectativas sobre o potencial das máquinas. O Teste de Turing, proposto em 1950 ([TURING, 1950](#)), estabeleceu critérios para avaliar se uma máquina poderia exibir comportamento inteligente indistinguível do humano, alimentando expectativas de que uma Inteligência Artificial Geral (AGI) estava próxima. Contudo, essas expectativas iniciais – por vezes desproporcionais aos resultados práticos – contribuíram para o surgimento do que viria a ser conhecido como o primeiro inverno da IA: um período marcado por investimentos reduzidos e ceticismo quanto às promessas tecnológicas. Essa fase de desaceleração, entretanto, impulsionou uma reavaliação dos métodos e estimulou novas abordagens, culminando em seu ressurgimento nas últimas décadas.

A arquitetura de Transformadores, introduzida em [2017](#), representa a base tecnológica de todos os modelos de linguagem de grande porte utilizados no *got.it*. Diferentemente de arquiteturas recorrentes anteriores, os Transformadores processam sequências de texto de forma paralela através de mecanismos de atenção, eliminando os gargalos sequenciais que limitavam o treinamento de modelos maiores.

O componente central da arquitetura de Transformadores é o mecanismo de atenção (*self-attention*), que permite ao modelo ponderar a relevância de diferentes partes da sequência de entrada ao processar cada *token*, e atualizar seu significado conforme entender a relevância dos mesmos. Formalmente, dado um vocabulário qualquer, pode-se definir representações sobre eles em posições específicas da entrada denominada de *token*, e o mecanismo calcula uma representação contextualizada desse *token* considerando sua relação com todos os outros *tokens* da sequência. Esta capacidade de capturar dependências de longo alcance sem limitações de distância é particularmente valiosa para o processamento de textos, onde conceitos relacionados podem estar separados por vários parágrafos mas precisam ser compreendidos em conjunto. O mecanismo de atenção opera através de múltiplas "cabecas" paralelas (*multi-head attention*), cada uma especializada em capturar diferentes tipos de relações semânticas e sintáticas, permitindo que o modelo mantenha representações ricas do texto.

Após o processamento pelo mecanismo de atenção, essa arquitetura contém ca-

camadas *feed-forward*, que consistem em redes neurais densamente conectadas aplicadas independentemente a cada posição da sequência. Estas camadas introduzem transformações não-lineares que permitem ao modelo aprender representações mais abstratas e complexas do conteúdo. A separação entre o mecanismo de atenção (que modela relações entre *tokens*) e as camadas *feed-forward* (que transformam as representações individuais) contribui para a capacidade de armazenamento de conhecimento e padrões linguísticos. No contexto do *got.it*, as camadas *feed-forward* dos modelos Qwen 3 4B e GPT-OSS 20B foram expostas durante o treinamento a vastos volumes de texto técnico e acadêmico, permitindo que capturem padrões linguísticos e conteúdos específicos deste domínio.

Para viabilizar a aplicação prática destes modelos de linguagem em um domínio específico como notas acadêmicas, o projeto emprega *LoRA* (*Low-Rank Adaptation*), uma técnica de *fine-tuning* que busca ser eficiente em parâmetros detalhada na Subseção 2.4.3.1. *LoRA* permite adaptar modelos pré-treinados injetando matrizes de baixo posto nas camadas de atenção, atualizando apenas uma fração dos parâmetros totais durante o treinamento.

2.4 Técnicas para construção de aplicações com LLMs

Apesar dos avanços recentes, desenvolver produtos com LLMs ainda se mostra um desafio, principalmente pela característica não-determinística e não-explicável dos modelos. Isso torna vantajoso usar algumas técnicas específicas para construir aplicações com LLMs, como as descritas nas subseções seguintes, usadas ao longo do desenvolvimento do projeto.

2.4.1 Retrieval-Augmented Generation (RAG)

Uma das limitações fundamentais dos LLMs em suas versões originais é a dificuldade de acessar informações que não estão presentes nos dados utilizados durante o treinamento. Para mitigar esse problema, algumas abordagens foram propostas. A mais direta consiste em inserir dados relevantes diretamente no contexto fornecido ao modelo durante a geração de respostas, prática conhecida como uso de *system prompts*. No entanto, simplesmente injetar mais informações não resolve o problema, por causa do limite de tamanho do contexto de *tokens* inerente à arquitetura dos Transformadores (isto é, o número de parâmetros em blocos de atenção cresce com o quadrado do número de *tokens* no contexto). Além disso, a acurácia dos modelos tende a diminuir significativamente quando o contexto aumenta muito (fenômeno conhecido como *needle in a haystack*), tornando difícil extrair informações específicas a partir das perguntas do usuário, fenômeno conhecido como *context rot* (HONG; TROYNIKOV; HUBER, 2024).

Este desafio motivou o desenvolvimento da abordagem conhecida como *Retrieval Augmented Generation* (RAG), que integra a recuperação de dados atualizados à geração

de respostas, possibilitando o enriquecimento das respostas com dados fora do treinamento, sem necessidade de fazer refinamentos adicionais nesses modelos.

2.4.1.1 Funcionamento do RAG

Os sistemas de *Retrieval-Augmented Generation* visam incorporar informações externas relevantes ao *prompt* de consulta do usuário. Para tanto, a consulta é projetada em um espaço vetorial, por meio de modelos de representação semântica, produzindo *embeddings* de alta dimensionalidade que capturam seu conteúdo semântico. Em seguida, essas representações são usadas para recuperar passagens semanticamente similares em uma base de conhecimento. Em seguida, os trechos obtidos são inseridos no contexto de entrada do modelo de linguagem de grande porte (*LLM*), que gera a resposta condicionada por essas informações.

2.4.2 Tool-Use

Em sistemas de IA usando LLMs, o uso de ferramentas – conhecido como *tool-use* – é implementado com o objetivo de transformar os modelos de linguagem em **agentes**, no sentido clássico da área: entidades que *percebem* um ambiente, *deliberam* e *agem* sobre ele.

O *ambiente* é definido como tudo o que está fora do agente e é relevante para a tarefa, incluindo fontes digitais (como *web*, bancos de dados, aplicações corporativas, sistemas operacionais, navegadores, serviços de mensageria), sistemas físicos (sensores e atuadores em robótica) e até outros agentes. As *ferramentas* são os meios pelos quais o agente observa e interage com esse ambiente: podem envolver consultas a APIs e bases de dados, execução de cálculos, chamadas a serviços especializados, leitura e escrita em sistemas de informação, navegação controlada pela *web* ou interações mediadas por usuários (por exemplo, sugerir e registrar ações conforme preferências do usuário).

O ciclo resultante é denominado de "perceber-pensar-agir": o modelo recebe *perceptos* do ambiente via ferramentas, raciocina e escolhe ações que maximizem uma medida de desempenho definida para a tarefa, especificada por um *PEAS* (*Performance, Environment, Actuators, Sensors*), conforme a formulação clássica. Na prática, essa agencialidade pode ser habilitada só com orquestração e instrução (por exemplo, usando padrões como *ReAct*, que intercalam raciocínio com ações). Quando necessário pode-se ainda complementar com aprendizado adicional 2.4.3 para decidir quando e como chamar ferramentas. Plataformas atuais oferecem *function/tool calling* com saídas estruturadas, conectando LLMs com dados e efeitos no mundo, de forma tipada, auditável e controlável. Com isso, os modelos deixam de depender apenas do conhecimento estático de pré-treinamento e passam a integrar informações atualizadas e operações precisas no próprio laço de geração de respostas (RUSSELL; NORVIG, 2021; SUTTON; BARTO, 2018; YAO et al., 2023; OPENAI, 2025).

2.4.3 Fine-Tuning

O *fine-tuning* é uma técnica que permite a alteração de determinados parâmetros de um modelo de linguagem de grande porte, após o seu treinamento. Ela é principalmente utilizada para transformar modelos generalistas em modelos especializados para um determinado domínio. A aplicação dessa técnica permite duas vantagens principais: a primeira é a maior acurácia em domínios específicos, e a segunda é diminuir o custo computacional do modelo.

O aumento da acurácia é resultado direto do pós-treinamento do modelo, que permite ampliar seu conhecimento sobre o ambiente específico em que será utilizado. Especificamente, pode-se treinar modelos para que conheçam previamente as ferramentas disponíveis com os parâmetros corretos. Intuitivamente, isso reduz certa carga cognitiva do modelo, tornando-o mais adaptado ao ambiente em que será avaliado.

A segunda vantagem, relacionada à redução dos custos computacionais, decorre da especialização do modelo para um domínio específico. Isso diminui a necessidade de generalização do modelo, permitindo o uso de modelos menores sem perdas significativas de acurácia. Por outro lado, modelos maiores e especializados tendem a oferecer acurácia superior, porém com maior latência e consumo de recursos, o que pode ser inadequado para aplicações que exigem respostas rápidas ou operam sob restrições de infraestrutura. Portanto, a escolha do modelo deve considerar o equilíbrio entre desempenho, custo computacional e requisitos da aplicação.

2.4.3.1 Métodos de Fine-Tuning

Há dois caminhos principais para implementar o *fine-tuning* em LLMs: o ajuste completo (*full fine-tuning*) e o ajuste eficiente em parâmetros (*parameter-efficient fine-tuning*, PEFT). O primeiro atualiza todos (ou a maior parte) dos pesos do modelo, com maior custo de memória e armazenamento. Já o segundo mantém os pesos originais “congelados” e treina apenas pequenos módulos adicionais, retendo desempenho com muito menos parâmetros treináveis (HOULSBY et al., 2019).

Entre os métodos de PEFT, destaca-se o LoRA (*Low-Rank Adaptation*). A ideia central do LoRA é que, em vez de atualizar todos os pesos de uma camada do modelo, é possível representar essas atualizações por meio de matrizes menores. Isso é feito decompondo a matriz de atualização em duas matrizes de dimensões reduzidas, o que diminui drasticamente o número de parâmetros treináveis e o consumo de memória. Ao final do treinamento, essas atualizações podem ser incorporadas aos pesos originais sem adicionar latência na inferência. Em Transformers, o LoRA é comumente aplicado às camadas de atenção, preservando desempenho próximo ao do ajuste completo com custos muito menores (HU et al., 2021).

Por fim, o *fine-tuning* pode ser orientado por instruções (*instruction tuning*), isto é, treinar com pares entrada-saída descritos em linguagem natural. Esse procedimento melhora a capacidade do modelo de seguir instruções e, em muitos casos, permite que modelos finamente ajustados alcancem desempenho de *zero-shot* superior ao de modelos maiores não ajustados (WEI et al., 2022).

2.5 Aspectos de Aplicação

Define-se *aplicação* como um sistema de *software* que oferece funções e uma interface para o usuário realizar certos objetivos (tarefas), geralmente usando outros sistemas por baixo (banco de dados, serviços, etc.). No contexto do *got.it*, a aplicação é o sistema que oferece as funções de criação, edição, e visualização de notas, e a interface é a forma como o usuário interage com o sistema. O projeto foi dividido em dois serviços principais, a interface, que é o meio pelo qual o usuário interage com o sistema, e o *backend*, que é o sistema que oferece as funções de criação, edição, e visualização de notas.

2.5.1 Aspectos de Backend

A tecnologia escolhida para o desenvolvimento do *backend* foi trocada algumas vezes ao longo do projeto. A primeira abordagem utilizou Node.js com TypeScript, servido pelo Express. Buscando otimizar o desenvolvimento, a equipe migrou para o *framework* Ruby on Rails, mais conhecido pelos integrantes.

Contudo, a implementação da persistência para as anotações do editor Tiptap apresentou desafios inesperados em Rails, devido à falta de suporte nativo. Diante dessa dificuldade, optou-se pela migração final para a plataforma Convex. Essa ferramenta agilizou o desenvolvimento do *backend* ao abstrair a complexidade da comunicação em tempo real e ao permitir a execução de funções assíncronas em TypeScript, cujos resultados são refletidos instantaneamente na interface.

A adoção do Convex permitiu iterar muito mais rapidamente sobre o projeto, facilitando a construção tanto da interface quanto do sistema de *backend* da aplicação.

2.5.2 Aspectos de Interface

Por se tratar de uma aplicação *web*, o *got.it* empregou tecnologias amplamente adotadas no desenvolvimento *web*, como TypeScript, React, Tailwind CSS, Shadcn UI e Vite.

Dentre as tecnologias escolhidas para o desenvolvimento da interface, o pacote Tiptap – baseado no Prosemirror – foi fundamental para a construção do editor de texto na *web* de forma flexível. O modelo do Prosemirror de "documento" representa o conteúdo

como uma árvore imutável de nós definida por um *schema* rígido, garantindo que o editor trabalhe sempre sobre estruturas válidas. O estado do editor reúne o documento, a seleção, marcas temporárias e os *plugins* ativos, sendo atualizado exclusivamente por transações, que descrevem qualquer mudança realizada. A "seleção" é um conceito unificado que pode representar tanto um cursor quanto um intervalo de texto ou até a seleção de um nó inteiro, permitindo que a lógica do editor trate diferentes casos de maneira consistente.

As transformações operam sobre o documento através de *steps* atômicos, como inserir, remover ou reestruturar conteúdo, possibilitando funcionalidades como histórico e colaboração. Por fim, a visualização conecta esse estado ao DOM, renderizando o conteúdo e interpretando eventos do usuário, enquanto *plugins* e decorações estendem o comportamento do editor sem alterar o documento em si. um *framework* de edição estruturada, composto por diversos módulos independentes que controlam o estado, a visualização, o modelo do documento e as transformações.

O Tiptap é uma camada acima do Prosemirror, em termos de abstração, facilitando aspectos comuns de editores de texto, como a criação de *plugins*, a integração com o React, e a estilização da interface, além de oferecer bibliotecas com funcionalidades comuns a serem utilizadas, como parágrafos, títulos, listas, entre outros.

A lista completa com os demais pacotes está disponível no repositório do código-fonte do projeto.

2.6 Aspectos de Infraestrutura

Define-se *infraestrutura* como o conjunto de tecnologias e serviços necessários para executar uma aplicação. Como um dos objetivos do projeto é ser *open-source*, as tecnologias utilizadas possuem licenças compatíveis com esse modelo. A infraestrutura foi projetada para simular um ambiente de produção real, incluindo mecanismos de redundância e monitoramento.

2.6.1 Orquestração e Implantação

Para a execução dos serviços do *got.it*, adotou-se o Kubernetes (*k8s*), um orquestrador de contêineres considerado padrão da indústria para sistemas em produção. O Kubernetes foi escolhido por oferecer alta disponibilidade, escalonamento automático de recursos e um ecossistema maduro de ferramentas complementares. Assim, é possível garantir que a aplicação permaneça disponível mesmo em caso de falhas em máquinas individuais.

A implantação de novas versões da aplicação é automatizada pelo ArgoCD, uma ferramenta de *continuous delivery* (CD) baseada em GitOps. Com essa abordagem, o

estado desejado do sistema fica versionado em um repositório Git, e o ArgoCD reconcilia automaticamente o *cluster* com esse estado. Isso elimina a necessidade de intervenção manual para realizar atualizações, reduzindo erros humanos e acelerando o ciclo de desenvolvimento.

2.6.2 Armazenamento e Persistência

A persistência de dados é garantida pelo Longhorn, uma solução de armazenamento distribuído para Kubernetes. Ele foi escolhido por oferecer replicação automática dos dados entre múltiplas máquinas e *backups* incrementais para armazenamento externo. Essa redundância é necessária para o sistema, uma vez que a perda de anotações dos usuários seria um evento catastrófico, podendo representar inúmeras horas de trabalho perdido.

O banco de dados relacional utilizado é o PostgreSQL, operado em configuração de alta disponibilidade com réplicas automáticas. O Postgres foi escolhido por ser uma das bases de dados mais utilizadas em ambientes de produção, com longo histórico de auditoria de segurança e altas garantias de retenção de dados. A configuração com réplicas garante continuidade de serviço mesmo em caso de falha do servidor principal.

2.6.3 Rede e Hospedagem

Como a infraestrutura é executada em servidores próprios (*bare metal*), adotou-se o MetalLB para expor os serviços na rede. Diferentemente de ambientes em nuvem, onde o provedor fornece balanceadores de carga automaticamente, em servidores próprios é necessário configurar essa funcionalidade manualmente. O MetalLB aloca endereços IP e os anuncia na rede, permitindo que clientes externos acessem os serviços do *cluster*.

A hospedagem pública da plataforma utiliza o Cloudflare Zero Trust em conjunto com o NGINX como *Ingress Controller*. O Cloudflare foi adotado para reduzir a superfície de ataque, oferecendo proteção integrada contra DDoS e *firewall* de aplicação *web* (WAF). O gerenciamento de certificados TLS é automatizado pelo *cert-manager* com emissão via Let's Encrypt, garantindo comunicação criptografada sem custo adicional.

2.6.4 Monitoramento

O monitoramento do sistema foi estruturado em duas frentes. Para a saúde operacional da infraestrutura, adotou-se o Prometheus para coleta de métricas e o Grafana para visualização em *dashboards*. Essa combinação permite acompanhar indicadores como utilização de CPU, memória e disponibilidade dos serviços.

Para o acompanhamento do comportamento dos usuários, utilizou-se o PostHog. Essa ferramenta permite instrumentar eventos de uso, mapear jornadas e identificar gargalos de usabilidade, orientando ciclos de melhoria contínua da plataforma.

2.6.5 Versionamento de Código

Adotou-se uma estratégia híbrida de hospedagem de repositórios, combinando GitHub e GitLab. O GitHub foi mantido para maximizar a visibilidade dos componentes *open-source* e facilitar a colaboração externa. Já o GitLab, implantado internamente no *cluster* Kubernetes, foi utilizado para automação de CI/CD e construção de imagens de contêineres, garantindo a reprodutibilidade do sistema de infraestrutura para qualquer pessoa interessada em continuar com o desenvolvimento da plataforma.

3 Método do trabalho

Este capítulo descreve a metodologia e as abordagens adotadas para o desenvolvimento do *got.it*, detalhando as etapas de levantamento de requisitos, planejamento ágil e técnicas específicas para a construção da aplicação.

3.1 Levantamento de requisitos via *Lean Inception*

Ao longo das reuniões regulares da equipe de trabalho, e após revisões coordenadas com o docente orientador, foram obtidos os seguintes resultados para as etapas do *Lean Inception*.

3.1.1 Visão do Produto

*"Para pessoas que querem aprender ou têm dificuldade de estudar de forma organizada, o **got.it** é um site de estudos que auxilia na construção de conhecimentos dos estudantes com a ajuda de Inteligência Artificial. Diferentemente do Notion, Obsidian, Evernote, ChatGPT, OneNote, Anki, o nosso produto é open-source, usa IA e tem um programa de aprendizado contínuo."*

3.1.2 É - Não é - Faz - Não faz

A Tabela 1 apresenta um quadro classificatório do produto, detalhando o que ele é, o que não é, o que faz e o que não faz. Isso ajuda a limitar o escopo e definir as funcionalidades do projeto, tanto para a equipe de desenvolvimento quanto para os usuários finais.

Tabela 1 – Quadro classificatório do produto.

É	Não é
• <i>Open-source</i> • Site responsivo • Gratuito • Simples de usar • Funcional em dispositivos móveis (<i>mobile</i>) • Intuitivo • Flexível	• Aplicativo nativo <i>desktop</i> • Plataforma de armazenamento • Substituto para didática • Notion, Obsidian, OneNote • Multi-usuário • Fonte de verdade/Pesquisa • Site de desenho • Site de IA (ex.: ChatGPT) • Aprendizado passivo
Faz	Não faz
• Ajuda na tomada de notas • <i>Chat</i> com suas notas • Organiza anotações • <i>Hyperlinks</i> automáticos • Sugestões contextuais • Perguntas e respostas • Cria planos de estudo • Anotações com <i>Markdown</i> • <i>Flashcards</i> automáticos	• Verificação completa do conteúdo (pode conter erros) • Pesquisar tópicos na <i>internet</i> • Resolução de operações matemáticas complexas

Fonte: Elaborado pelos autores.

3.1.3 Descrição das personas

Nesta etapa, a equipe de trabalho investigou diversos perfis de estudantes que poderiam se beneficiar com o uso do projeto. Foram identificadas três *personas* principais, cada uma representando um perfil distinto de usuário:

- *Estudante de Economia*: representa usuários que precisam organizar grandes volumes de conteúdo teórico e conceitual, com foco em anotações estruturadas e revisão sistemática de materiais.
- *Estudante de Engenharia de Computação*: representa usuários com perfil técnico, familiarizados com ferramentas digitais, que valorizam funcionalidades como formatação em *Markdown*, *hyperlinks* e integração com fluxos de trabalho ágeis.
- *Estudante de Medicina*: representa usuários que necessitam memorizar extenso vocabulário técnico, priorizando funcionalidades como flashcards automáticos e repetição espaçada para retenção de conhecimento.

A análise dessas *personas* permitiu identificar características, comportamentos e necessidades distintas, verificando quais recursos cada perfil mais valorizaria. A descrição detalhada de cada *persona* encontra-se no Apêndice A.

3.1.4 Jornadas dos Usuários

A seguir, apresenta-se um fluxograma exemplificando a jornada de um usuário típico, conforme uma das *personas* identificadas. A Figura 2 ilustra as principais interações do usuário com o sistema ao realizar uma tarefa, para um estudante de Economia (hipotético). Outros fluxogramas estão no Apêndice B.

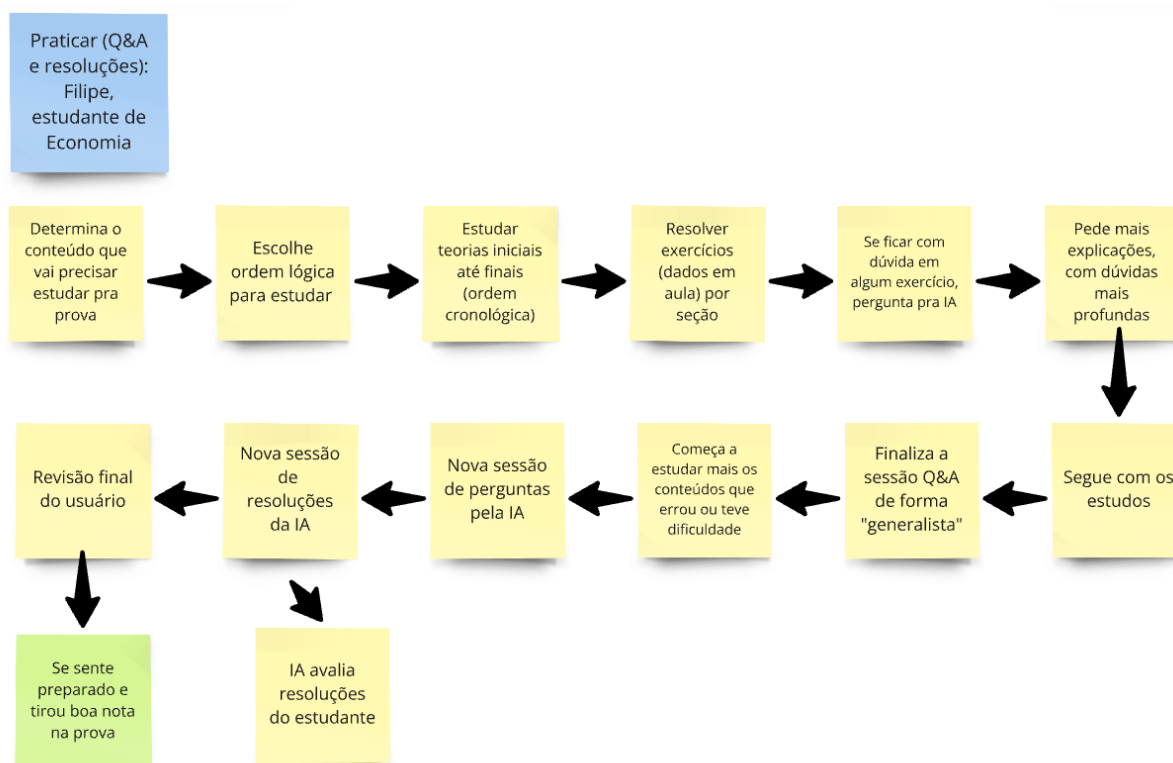


Figura 2 – Descrição da jornada da prática de exercícios na revisão com o auxílio de IA.

Fonte: Elaborado pelos autores.

3.1.5 Brainstorming de funcionalidades

Nesta etapa, a equipe de trabalho levantou as funcionalidades a serem incluídas no projeto. Elas são descritas de forma simples e genérica inicialmente, como um substantivo ou na forma "verbo + objeto".

3.1.6 Revisão Técnica, de UX e de Negócios das Funcionalidades

Cada uma das funcionalidades foi classificada de acordo com base nos parâmetros descritos:

1. **Esforço:** Percepção do trabalho que será requerido pela equipe para implementar a funcionalidade. Pode apresentar de 1 a 3 pontos e é identificado com a letra "E" maiúscula.

2. **Negócio:** Nível do valor que será entregue com a finalização dessa funcionalidade. Varia de 1 a 3 pontos (cujo símbolo é o cifrão "\$").
3. **UX:** Nível do valor percebido pelos usuários com o uso da funcionalidade. Varia de 1 a 3 pontos (representados pelos corações: "< 3").
4. **Nível de confiança:** Estimativa da certeza em que a equipe tem em desenvolver a funcionalidade, no que diz respeito ao *que* precisará ser feito, e ao *como*. A classificação foi feita em três níveis: alta confiança (verde), média (amarelo) e baixa (vermelho).

3.1.7 Sequenciador

Nesta etapa, são definidas as "ondas" do *Lean Inception*, que representam as unidades de trabalho que irão compor o MVP, ou um conjunto de funcionalidades incremental completo.

As funcionalidades são priorizadas de acordo com o valor que proporcionam ao projeto e o nível de confiança da equipe em sua implementação, sendo alocadas nas ondas iniciais aquelas de maior impacto e viabilidade, conforme visto na Figura 3.

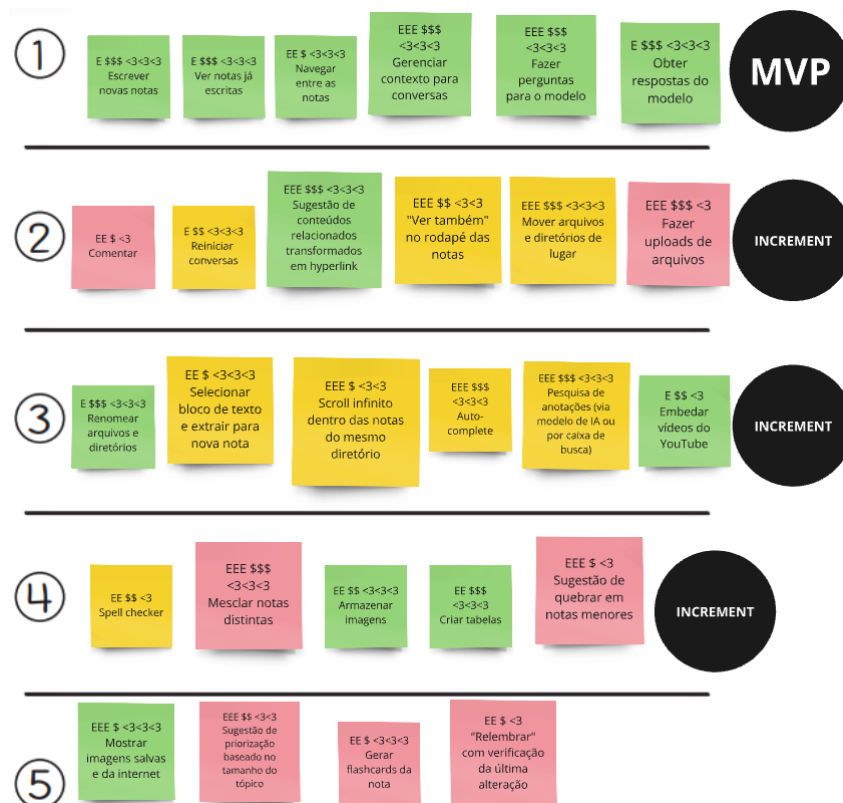


Figura 3 – Funcionalidades agrupadas e classificadas, resultado das etapas 3.1.5, 3.1.6 e 3.1.7.

Fonte: Elaborado pelos autores.

3.1.8 Canvas MVP

Nesta seção, é apresentado o Canvas MVP, uma ferramenta utilizada no processo de *Lean Inception* para definir a versão inicial do produto que será entregue aos usuários. O Canvas MVP ajuda a estruturar e priorizar as funcionalidades essenciais do Mínimo Produto Viável (MVP), garantindo que a equipe esteja alinhada quanto ao escopo inicial e aos objetivos principais do produto. (CAROLI, 2018)

Inicialmente, descreve-se a estrutura do Canvas MVP, que inclui os seguintes elementos:

- **Personas segmentadas:** Quem são os principais usuários do produto?
- **Proposta do MVP:** O que o MVP se propõe a verificar.
- **Principais Funcionalidades:** Quais são as funcionalidades mínimas que o produto deve ter para validar sua proposta de valor?
- **Jornadas do usuário:** Descritas detalhadamente na etapa anterior, descreve as sequências de etapas a serem seguidas pelo usuário para obter uma determinada finalidade, contempladas pelo produto.
- **Custo e cronograma:** Estimativa dos recursos financeiros e tempo a serem despendidos no projeto.
- **Resultado esperado:** Quais suposições, ao longo de um período de demonstração, precisam ser validadas para garantir que o produto atenda às necessidades do mercado?
- **Métricas para validação das hipóteses de negócio:** Quais métricas serão utilizadas para medir o impacto do MVP?

Em seguida, aplicando os resultados das etapas anteriores, foram consolidadas as conclusões da equipe a respeito dos elementos que compõem o MVP, como ilustrado na Figura 4.

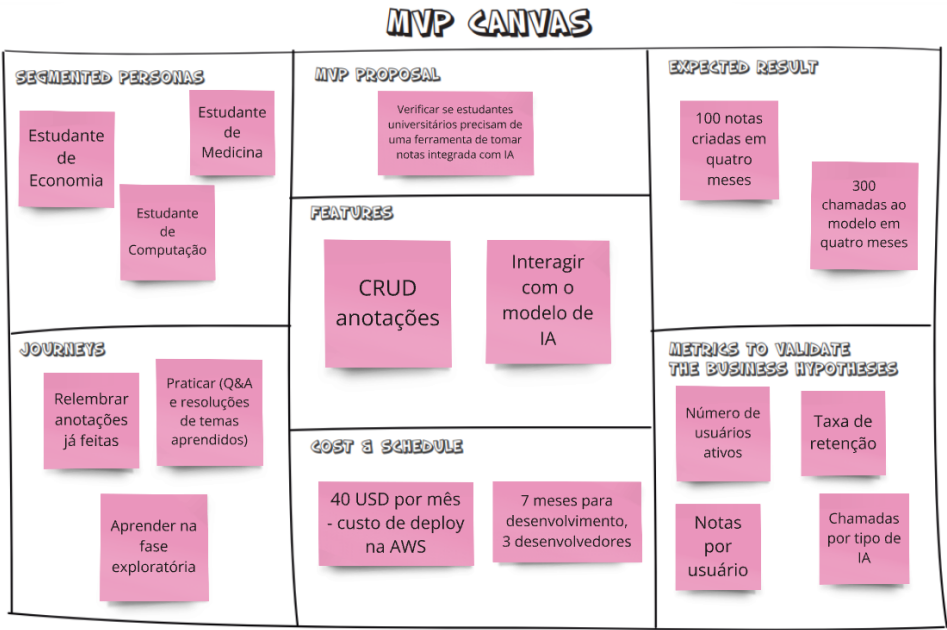


Figura 4 – Canvas do MVP.

Fonte: Elaborado pelos autores.

3.2 Processo de desenvolvimento

Nesta parte do trabalho, é apresentado o planejamento do desenvolvimento do projeto, detalhando como cada fase do processo se insere ao longo do ano letivo e sua duração estimada. Utilizou-se um cronograma visual (diagrama de Gantt) para representar a distribuição das tarefas ao longo do tempo, exibido abaixo.

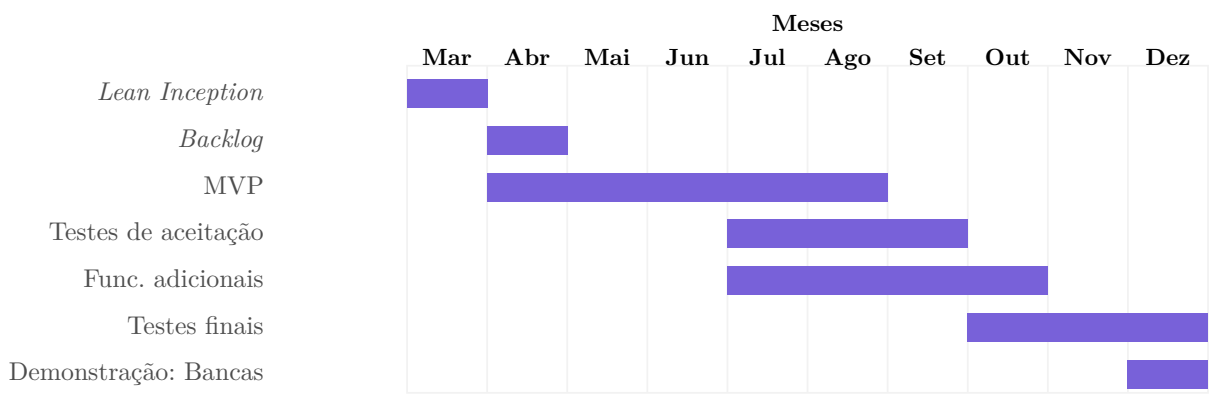


Figura 5 – Cronograma do projeto em diagrama de Gantt.

Fonte: Elaborado pelos autores.

4 Especificação de Requisitos

Para a especificação de requisitos, foi adotada uma abordagem ágil e colaborativa, que fundamentou-se no uso de "histórias de usuários" para capturar as necessidades e expectativas dos usuários finais como definido nas funcionalidades levantadas no sequenciamento. Essa abordagem possibilitou uma visão prática e centrada no usuário, facilitando a definição e priorização dos requisitos.

Apesar de não formalizar "critérios de aceite" de forma rigorosa, em razão do tamanho reduzido do grupo, a comunicação constante e o alinhamento entre os membros da equipe garantiram que dúvidas e desafios fossem rapidamente esclarecidos. Esse diálogo contínuo permitiu:

- **Validação rápida dos requisitos:** Por meio de reuniões periódicas e feedbacks informais, todos os membros puderam discutir e ajustar as funcionalidades conforme a evolução do projeto.
- **Flexibilidade e adaptabilidade:** A ausência de formalismos excessivos permitiu a reavaliação e adaptação dos requisitos ao longo do desenvolvimento, atendendo às mudanças e novas ideias que surgiam.
- **Alinhamento e comprometimento:** A comunicação direta facilitou a coordenação das atividades, garantindo que todos os integrantes do grupo compartilhassem uma visão comum sobre as funcionalidades a serem implementadas.

Essa metodologia se mostrou adequada para projetos de menor escala, em que a informalidade dos critérios de aceite é compensada pela proximidade e constante interação entre os membros da equipe, permitindo assim uma entrega contínua e alinhada com as necessidades do usuário.

4.1 MVP: Tomada de anotações e interação com o modelo

A definição de pronto para essa onda consiste nos testes dos recursos, que devem ser concluídos com sucesso, e a verificação dos padrões (*design patterns*) do código produzido. A primeira onda de funcionalidades, que compõe os recursos básicos do projeto, estão detalhados abaixo:

1. **Escrever novas notas:** Como estudante, quero ser capaz de escrever, formatar e inserir minhas notas no *got.it*, para auxiliar na exposição inicial a um assunto.

2. **Ver notas já escritas:** Como estudante, quero abrir o *got.it* e visualizar as notas salvas anteriormente da mesma maneira em que foram salvas, para continuar a tomada de anotações ou fazer revisões.
3. **Navegar entre as notas:** Como estudante, quero abrir o *got.it* e visualizar as notas na ordem e nas pastas em que elas foram inseridas, para poder visualizá-las.
4. **Gerenciar contexto para conversas:** Como estudante, quero que o contexto da LLM utilizada seja reiniciada com uma conversa nova, e que mensagens trocadas anteriormente na conversa sejam recordadas pelo assistente virtual, para que eu possa obter respostas mais elaboradas.
5. **Fazer perguntas para o modelo:** Como estudante, quero conseguir enviar para a LLM utilizada uma mensagem.
6. **Obter respostas do modelo:** Como estudante, quero receber uma resposta da LLM coerente com o *prompt* e o contexto da conversa após enviar uma mensagem, para que o assistente virtual possa auxiliar na geração de resumos, questões ou raciocínio.

4.2 Incremento 1: Recursos adicionais para as notas

Com as funcionalidades básicas de anotação e interação com o modelo de IA estabelecidas no MVP, este incremento foca em enriquecer a experiência de tomada de notas. Os recursos incluem ferramentas de organização, navegação entre conteúdos relacionados por meio de *hyperlinks* e a capacidade de gerenciar a estrutura de arquivos e diretórios. Essas adições visam facilitar a construção de um acervo de conhecimento interconectado.

1. **Comentar:** Como estudante, quero inserir comentários avulsos em minhas anotações, para que eu possa organizar meu raciocínio durante os estudos.
2. **Reiniciar conversas:** Como estudante, quero poder reiniciar um contexto de conversa com o assistente, para que eu possa iniciar um novo fluxo de raciocínio.
3. **Sugestão de conteúdos relacionados transformados em *hyperlink*:** Como estudante, quero visualizar links a conteúdos relacionados, para que eu possa navegar entre os tópicos durante os estudos.
4. **"Ver também" no rodapé das notas:** Como estudante, quero inserir tópicos relacionados nos rodapés das notas, para que eu possa navegar entre os tópicos durante os estudos.

5. **Mover arquivos e diretórios de lugar:** Como estudante, quero mover diretórios e arquivos na hierarquia, para que eu possa corrigir algum erro ou melhorar a estrutura das notas.
6. **Fazer *uploads* de arquivos:** Como estudante, quero poder fazer *uploads* de arquivos multimídia (texto, imagem, vídeo) para que eu possa ter acesso rápido a materiais suplementares.

4.3 Incremento 2: Recursos adicionais para a aplicação

Este incremento expande as capacidades da aplicação como um todo, introduzindo recursos de produtividade que otimizam o fluxo de trabalho do estudante. As funcionalidades incluem ferramentas de busca para localizar anotações, *auto-complete* para agilizar a digitação, e melhorias de navegação como *scroll* contínuo entre notas relacionadas.

1. **Renomear arquivos e diretórios:** Como estudante, quero renomear diretórios e arquivos, para que eu possa corrigir algum erro cometido ao nomeá-los.
2. **Selecionar bloco de texto e extrair para nova nota:** Como estudante, quero selecionar um trecho em uma nota existente e criar uma nova, para que eu possa reorganizar minhas anotações.
3. ***Scroll* infinito dentro das notas do mesmo diretório:** Como estudante, quero rolagem automática entre as notas em um mesmo diretório (conectadas logicamente), para que o fluxo de estudos seja ininterrupto.
4. ***Auto-complete*:** Como estudante, quero um recurso de sugestão para completar palavras, para que minha digitação seja mais ágil.
5. **Pesquisa de anotações (via modelo de IA ou por caixa de busca):** Como estudante, quero poder pesquisar palavras-chave de conceitos dentro do *got.it*, para que eu possa retomar uma anotação ou recuperar uma referência de estudo.
6. **Embedar vídeos do *YouTube*:** Como estudante, quero poder inserir *links* para vídeos do *YouTube* com suas imagens miniatura, para que eu possa ter referências a recursos de vídeo adicionais.

4.4 Incremento 3: Recursos avançados das notas

Com a estrutura da aplicação consolidada nos incrementos anteriores, este incremento adiciona recursos avançados de criação de conteúdo. As funcionalidades permitem a

elaboração de notas mais ricas e completas, com suporte a elementos visuais como imagens e tabelas, além de ferramentas de qualidade como verificação ortográfica.

1. **Verificador de erros ortográficos:** Como estudante, quero uma ferramenta de verificação e correção de erros, para que eu tenha menos erros ortográficos, e minhas anotações sejam mais polidas.
2. **Mesclar notas distintas:** Como estudante, quero unir duas notas relacionadas em uma única, para que eu possa reorganizar minhas anotações.
3. **Armazenar imagens:** Como estudante, quero poder salvar imagens para que as minhas anotações possam ter diagramas específicos que me ajudem a estudar.
4. **Criar tabelas:** Como estudante, quero poder criar tabelas nas anotações para ter uma melhor experiência de usuário.
5. **Sugestão de quebrar em notas menores:** Como estudante, quero visualizar um lembrete de criar notas novas quando a anotação atingir um tamanho muito grande, para facilitar minha organização.

4.5 Incremento 4: Recursos de priorização e planejamento de estudos

O incremento final foca em recursos de aprendizado ativo e planejamento de estudos. Integra técnicas comprovadas de retenção de conhecimento, como repetição espaçada por meio de *flashcards* gerados automaticamente, além de funcionalidades de priorização que auxiliam o estudante a identificar quais tópicos necessitam de revisão.

1. **Mostrar imagens salvas e da *internet*:** Como estudante, quero visualizar imagens armazenadas via *upload* ou referenciadas pelo seu endereço na *internet*.
2. **Sugestão de priorização baseado no tamanho do tópico:** Como estudante, quero saber quais tópicos eu preciso rever para ter o meu aprendizado em dia.
3. **Gerar *flashcards* da nota:** Como estudante, quero gerar *flashcards* da nota de um tópico específico, para que eu possa efetuar rapidamente a revisão.
4. **"Relembrar" com verificação da última alteração:** Como estudante, quero visualizar tópicos antigos, para que eles não sejam negligenciados no período de estudos.

4.6 Requisitos Não Funcionais

Esta seção apresenta os requisitos não funcionais do sistema, que definem características de qualidade para a experiência do usuário. Diferentemente dos requisitos funcionais que descrevem o que o sistema faz, estes requisitos especificam como o sistema deve se comportar em termos de disponibilidade, desempenho e proteção de dados.

1. **Alta disponibilidade do sistema:** Como equipe de operações, quero que o *got.it* continue funcionando mesmo com a perda de até 1 nó da infraestrutura, para que meus estudos não sejam interrompidos por falhas técnicas.
2. **Persistência e backup de dados:** Como estudante, quero que minhas anotações sejam salvas automaticamente a cada 30 segundos e protegidas por *backups* diários, para que eu não perca horas de trabalho, em caso de falhas.
3. **Resposta rápida do autocomplete:** Como estudante, quero que as sugestões de *autocomplete* apareçam em menos de 500 milissegundos, para que minha digitação não seja interrompida pela espera de respostas da IA.
4. **Comunicação segura:** Como estudante, quero que toda comunicação com o *got.it* seja criptografada via TLS, para que minhas anotações e dados pessoais estejam protegidos contra interceptação.
5. **Carregamento rápido da aplicação:** Como estudante, quero que o *got.it* carregue em menos de 3 segundos, para que eu possa começar a estudar sem esperas frustrantes.

4.7 Requisitos de Infraestrutura e Operações

Esta seção descreve os requisitos relacionados à infraestrutura e operações do sistema, sob a perspectiva da equipe técnica responsável pela manutenção da plataforma. Estes requisitos garantem que o sistema possa ser operado, monitorado e atualizado de forma eficiente e segura, sustentando os requisitos não funcionais apresentados na seção anterior. Destaca-se que esses requisitos foram desenvolvidos em conjunto com a primeira onda do MVP, sendo necessários para que o projeto fosse implantado em produção.

1. **Deploy automatizado via GitOps:** Como equipe de operações, quero que novas versões da aplicação sejam implantadas automaticamente a partir do repositório Git, para que as atualizações sejam consistentes e auditáveis.
2. **Monitoramento e alertas:** Como equipe de operações, quero visualizar métricas de saúde do sistema em tempo real e receber alertas sobre anomalias, para que problemas sejam identificados antes de impactarem os usuários.

3. **Rollback automático:** Como equipe de operações, quero que *deployments* com falhas sejam revertidos automaticamente para a versão anterior estável, para que usuários não sejam afetados por *releases* defeituosas.
4. **Escalabilidade horizontal:** Como administrador do sistema, quero que a aplicação possa escalar automaticamente o número de réplicas conforme a demanda, para que o sistema mantenha *performance* mesmo com aumento de usuários.
5. **Análise de comportamento:** Como equipe de operações, quero coletar e visualizar métricas de uso da plataforma, para que decisões de produto sejam baseadas em dados reais de comportamento dos usuários.

5 Desenvolvimento do Trabalho

Nesse capítulo descreve-se o processo de implementação do projeto. Serão descritos as tecnologias utilizadas, bem como o motivo pelo quais elas foram escolhidas, a arquitetura geral do projeto, além dos detalhes de implementação de cada uma das partes do sistema. Por fim, serão descritos os testes realizados para avaliar se o mesmo atende os requisitos de usabilidade e funcionalidade.

5.1 Projeto e Implementação

Esta seção busca apresentar o processo completo de desenvolvimento da plataforma *got.it*, desde a concepção arquitetural até o *deploy* em produção. O desenvolvimento seguiu uma abordagem incremental, partindo de um MVP (*Minimum Viable Product*) e evoluindo através de incrementos sucessivos, conforme a metodologia *Scrum* descrita no Capítulo 2. No início do desenvolvimento do projeto, percebemos que, devido a equipe ser muito enxuta (apenas 3 membros) e estar em constante contato via canais alternativos, certas práticas do *Scrum* não agregavam significativamente na qualidade do desenvolvimento, comparado com o seu custo de tempo. Portanto, tomou-se a decisão de conduzir o projeto de forma híbrida, adotando certas práticas do *Scrum* que eram vantajosas, mas abolindo outras mais ritualísticas.

5.1.1 Visão Geral da Arquitetura

5.1.1.1 Camada de Aplicação

A arquitetura da aplicação do *got.it* é estruturada em quatro camadas principais, cada uma com responsabilidades distintas e tecnologias específicas. A Figura 6 ilustra a organização geral da aplicação *web*, com o *backend*, o *frontend* e uma ferramenta externa chamada LM Studio, responsável por executar os LLMs no servidor.

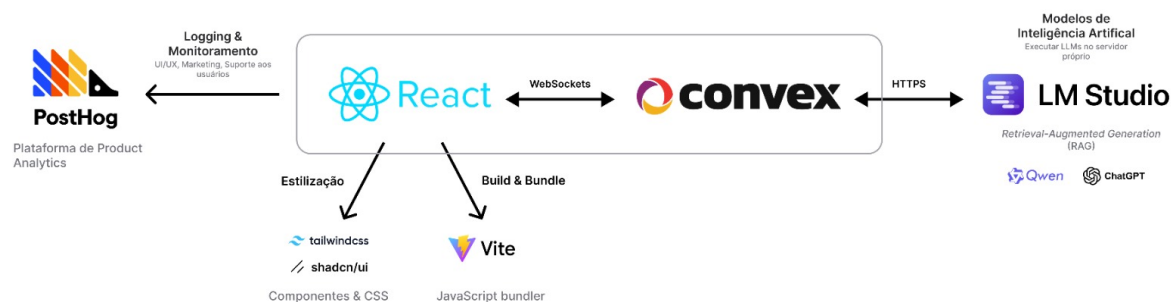


Figura 6 – Ilustração da aplicação *web* do *got.it*, mostrando as camadas principais e seus componentes.

Fonte: Elaborado pelos autores.

A figura apresenta os principais componentes da aplicação e suas interconexões. No centro, o React constitui o núcleo do *frontend*, comunicando-se com o Convex via WebSockets para sincronização em tempo real. O Convex, por sua vez, conecta-se ao LM Studio via HTTPS para executar os modelos de inteligência artificial Qwen e ChatGPT. À esquerda, o PostHog coleta dados de *logging* e monitoramento para análise de comportamento dos usuários. Na parte inferior, o Tailwind CSS e Shadcn/UI fornecem a estilização e os componentes visuais, enquanto o Vite realiza o *build* e *bundling* do código JavaScript.

A camada de *frontend* é responsável pela interface com o usuário e executa inteiramente no navegador do cliente. Desenvolvida com React e TypeScript – conforme detalhado na Subseção 2.5.2 – esta implementa toda a lógica de apresentação e interação, incluindo o editor de texto colaborativo baseado em Tiptap, os componentes de interface construídos com Shadcn UI estilizados via Tailwind CSS, e os mecanismos de gerenciamento de estado da aplicação. A escolha de uma arquitetura de *Single Page Application* (SPA) permite que a interface seja altamente responsiva e interativa. Essa abordagem é ideal para aplicações que, embora possam ter um tempo de carregamento inicial maior, se beneficiam de atualizações de interface em tempo real sem a necessidade de recarregar a página inteira. Após o processo de *build* realizado pelo Vite, o código do *frontend* é empacotado (*bundled*) em arquivos estáticos, que são então distribuídos pela infraestrutura de hospedagem.

A camada de *backend* gerencia toda a lógica de negócio, persistência de dados, autenticação e comunicação em tempo real. Ela é implementada utilizando o Convex, conforme explicado na Subseção 2.5.1, que oferece um modelo de desenvolvimento baseado em funções *serverless* escritas em TypeScript. O Convex abstrai a complexidade de implementar comunicação *real-time* através de *WebSockets*, fornecendo primitivas de alto nível como *queries* (consultas reativas que atualizam automaticamente quando os dados mudam), *mutations* (operações que modificam o estado) e *actions* (funções que podem

realizar efeitos colaterais como chamadas a APIs externas). Esta arquitetura orientada a funções facilita significativamente o desenvolvimento, permitindo que modificações no *backend* sejam implementadas rapidamente sem a necessidade de gerenciar explicitamente servidores, *endpoints* REST ou protocolos de comunicação em tempo real. O Convex também gerencia automaticamente a persistência dos dados e o cacheamento de dados acessados frequentemente, oferecendo garantias transacionais e latência muito reduzida, sem a necessidade de implementação manual de uma camada de *cache*.



Figura 7 – Fluxo de trabalho do Git adotado no projeto, ilustrando a estratégia de versionamento e CI/CD.

Fonte: Elaborado pelos autores.

A gestão do código-fonte do projeto adota uma estratégia de duplo repositório, com o código versionado simultaneamente no GitHub e no GitLab, conforme ilustrado na Figura 7. A utilização do GitLab alinha-se ao objetivo de manter o projeto em uma plataforma de modelo *open-core*, refletindo a natureza *open-source* do *got.it*. No entanto, por questões de familiaridade da equipe e conveniência operacional, o GitHub foi definido como o repositório principal para o desenvolvimento diário, com sincronizações periódicas garantindo a paridade entre as duas plataformas. Para os processos de integração e entrega contínua (*continuous integration* e *continuous deployment*, ou CI/CD), foram utilizados os GitLab Pipelines para automatizar os testes e *build* da aplicação.

Por fim, a camada de observabilidade da aplicação é implementada usando o PostHog, uma plataforma de *product analytics* utilizada para monitorar o engajamento e o comportamento dos usuários. A integração do PostHog permitiu à equipe acompanhar o uso do *got.it* em produção, capturando eventos, monitorando funis de conversão e identificando erros de *frontend*. Este acompanhamento detalhado do comportamento do usuário é fundamental para a melhoria contínua da aplicação.

5.1.1.2 Camada de Infraestrutura

A infraestrutura do *got.it* foi implantada em um *cluster* Kubernetes, uma plataforma de orquestração de contêineres que automatiza a implantação, o escalonamento e o gerenciamento de aplicações. Para compreender as decisões arquiteturais apresentadas a seguir, é necessário apresentar brevemente os conceitos fundamentais do Kubernetes.

5.1.1.2.1 Conceitos Básicos do Kubernetes

O Kubernetes organiza as cargas de trabalho em três abstrações principais. Um *Pod* é a unidade mínima de implantação, que agrupa um ou mais contêineres que compartilham recursos de rede e armazenamento. Um *Service* é uma abstração que expõe um conjunto de *Pods* como um *endpoint* de rede estável, permitindo a comunicação entre componentes sem depender de endereços IP efêmeros. Por fim, um *Deployment* gerencia a criação e atualização de réplicas de *Pods*, garantindo que o número desejado de instâncias esteja sempre em execução.

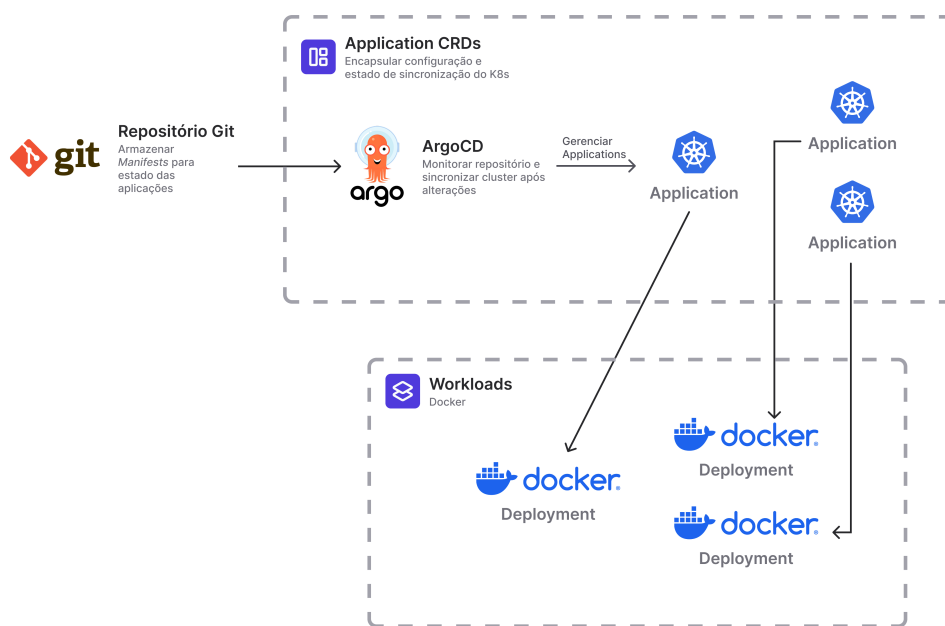


Figura 8 – Arquitetura básica do Kubernetes

Fonte: Elaborado pelos autores.

A Figura 8 ilustra a organização de um *cluster* Kubernetes. No topo, o *Control Plane* contém o *API Server*, ponto central de comunicação que coordena todas as operações. Os *Agent Nodes* executam os *Pods* através do Kubelet, um agente que garante que os contêineres estejam em execução conforme especificado. O *Deployment*, representado como uma abstração, gerencia réplicas de *Pods* distribuídas entre os nós, enquanto o *Service* atua como ponto de entrada estável, roteando o tráfego para os *Pods* apropriados independentemente de sua localização física no *cluster*.

5.1.1.2.2 Redes

O Kubernetes abstrai a complexidade de rede através de um modelo onde cada *Pod* recebe um endereço IP único dentro do *cluster*, e os *Services* atuam como balanceadores

de carga internos com resolução DNS automática. No entanto, para expor aplicações ao mundo externo, são necessários componentes adicionais.

O Nginx Ingress Controller atua como ponto de entrada para o tráfego HTTP/HTTPS, roteando requisições para os *Services* apropriados com base em regras de *host* e caminho. Ele funciona como um proxy reverso configurável via recursos declarativos do Kubernetes.

Para expor os serviços à Internet sem a necessidade de *port-forwarding* ou endereços IP públicos, foi utilizado o Cloudflare Zero Trust. Esta solução estabelece um túnel seguro e criptografado entre o *cluster* e a infraestrutura da Cloudflare, permitindo que o tráfego externo alcance o Nginx Ingress Controller de forma segura, sem expor diretamente os nós do *cluster*.

A automatização da geração e renovação dos certificados SSL foi implementada através do Cert-Manager, um controlador que gerencia o ciclo de vida dos certificados TLS. Ele provisiona certificados automaticamente junto a autoridades certificadoras como Let's Encrypt e os renova antes da expiração, eliminando a necessidade de intervenção manual.

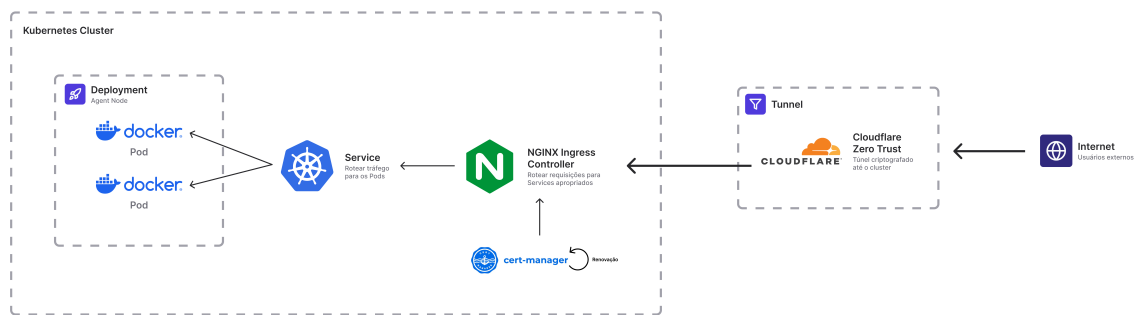


Figura 9 – Arquitetura de rede do *cluster*

Fonte: Elaborado pelos autores.

A Figura 9 apresenta o fluxo de tráfego desde a *Internet* até os *Pods*. As requisições dos usuários passam primeiro pelo Cloudflare Zero Trust, que estabelece um túnel criptografado até o *cluster*. O tráfego é então recebido pelo Nginx Ingress Controller, que roteia as requisições para os *Services* apropriados com base nas regras de *host* e caminho configuradas. Os *Services*, por sua vez, encaminham o tráfego para os *Pods* correspondentes. O Cert-Manager opera em paralelo, garantindo que os certificados TLS estejam sempre válidos e atualizados.

5.1.1.2.3 Persistência de Dados

O Kubernetes foi projetado para ser extensível, permitindo que fornecedores implementem suas próprias soluções de armazenamento através da especificação CSI (*Container*

Storage Interface). Esta interface padronizada permite que o *cluster* provisione volumes persistentes de diferentes sistemas de armazenamento de forma uniforme.

Para o provisionamento de volumes persistentes distribuídos, foi utilizado o Longhorn. Esta solução replica os dados em pelo menos três nós do *cluster*, garantindo proteção contra falhas de *hardware* ou corrupção de um dos volumes. Além da replicação síncrona, o Longhorn oferece *snapshots* e *backups* incrementais, que podem ser configurados para garantir políticas de retenção de dados.

O banco de dados PostgreSQL foi provisionado em uma configuração de alta disponibilidade através do operador CloudNativePG. Esta configuração utiliza três réplicas distribuídas: uma instância primária responsável pelas operações de escrita e duas réplicas de leitura. O operador monitora continuamente a saúde das instâncias e, em caso de falha do nó primário, promove automaticamente uma das réplicas sem necessidade de intervenção manual, assegurando a continuidade da operação.

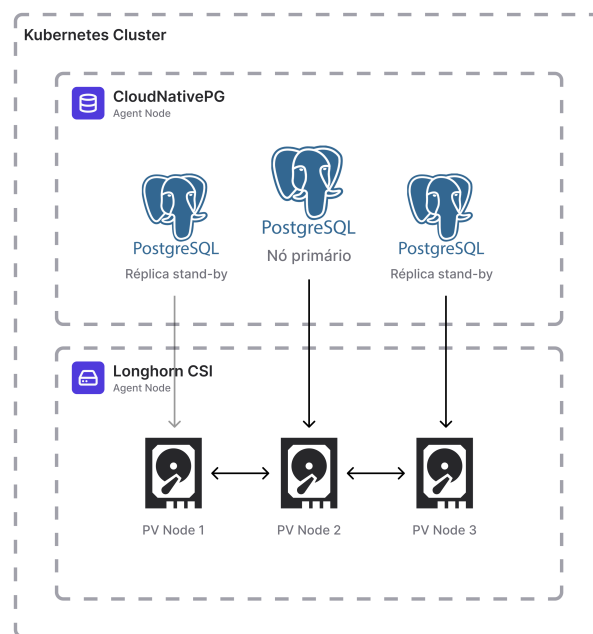


Figura 10 – Arquitetura de persistência de dados

Fonte: Elaborado pelos autores.

A Figura 10 ilustra a arquitetura de persistência implementada. Na camada superior, o CloudNativePG gerencia o PostgreSQL em alta disponibilidade, com uma instância primária que aceita operações de leitura e escrita, e duas réplicas que recebem as atualizações de forma síncrona. Cada instância do banco de dados utiliza um *PersistentVolume* provisionado pelo Longhorn, que replica os dados entre três nós distintos do *cluster*. Esta dupla camada de replicação (tanto no nível do banco de dados quanto no nível do

armazenamento) garante alta disponibilidade e proteção contra perda de dados.

5.1.1.2.4 Aplicações e Entrega Contínua

O ArgoCD implementa o paradigma GitOps, onde o estado desejado das aplicações é declarado em um repositório Git e o *cluster* é continuamente reconciliado para refletir esse estado. No Kubernetes, o ArgoCD estende a API através de *Custom Resource Definitions* (CRDs), permitindo que aplicações sejam gerenciadas como recursos nativos do *cluster*.

Esta abordagem declarativa traz diversos benefícios: o histórico de mudanças é preservado no controle de versão, permitindo auditoria completa; *rollbacks* são realizados simplesmente revertendo um *commit*; e múltiplos ambientes podem ser gerenciados de forma consistente. O ArgoCD detecta automaticamente novas versões no repositório e realiza os *deployments*, aplicando estratégias de *rollback* automático caso a nova versão apresente falhas, reduzindo o risco de erros no processo de entrega contínua.

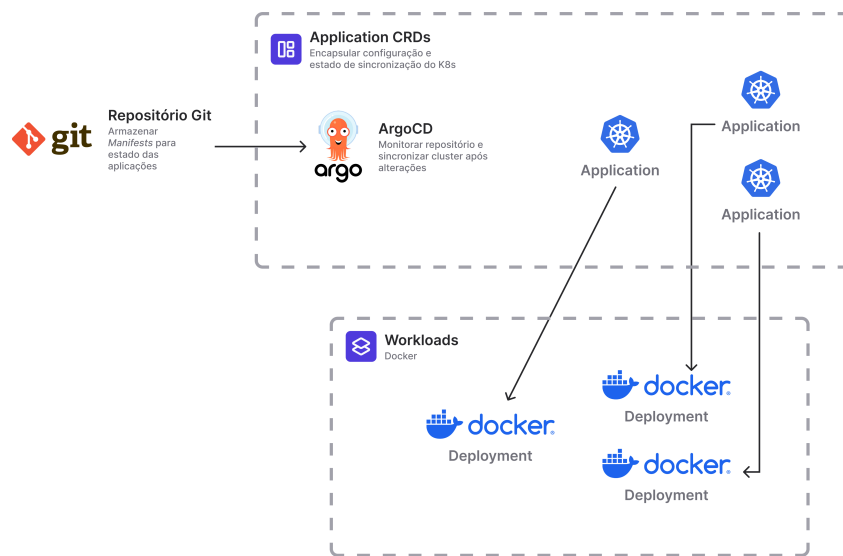


Figura 11 – Fluxo GitOps com ArgoCD

Fonte: Elaborado pelos autores.

A Figura 11 demonstra o fluxo GitOps implementado. O repositório Git armazena os *manifests* declarativos que definem o estado desejado das aplicações. O ArgoCD monitora continuamente este repositório e, ao detectar alterações, sincroniza o estado do *cluster* para corresponder às definições declaradas. Cada aplicação é representada como um *Application CRD*, um recurso nativo do Kubernetes que encapsula a configuração e o estado de sincronização. Estes recursos, por sua vez, gerenciam os *workloads* correspondentes (*Deployments*, *Services* e demais recursos necessários para a execução da aplicação).

5.1.1.2.5 Monitoramento e Observabilidade

A observabilidade do *cluster* é necessária para diagnosticar problemas e tomar decisões informadas sobre disponibilidade e desempenho. Para isso, foi implementado o Prometheus, um sistema de monitoramento que coleta métricas de todos os componentes do *cluster*, armazenando-as em um banco de dados de séries temporais.

A visualização das métricas coletadas é realizada através do Grafana, uma plataforma que permite criar *dashboards* personalizados e configurar alertas. Juntos, Prometheus e Grafana permitem o rápido diagnóstico de problemas de *hardware*, rede ou aplicação, fornecendo visibilidade sobre o estado do sistema em tempo real.

5.1.1.2.6 Deploy das Aplicações

O *frontend* e o *backend* do *got.it* seguem o mesmo padrão de implantação no *cluster*. Cada aplicação é definida como um *Deployment*, que gerencia as réplicas dos *Pods* distribuídas entre os nós disponíveis. Um *Service* expõe cada *Deployment* internamente, e o Nginx Ingress Controller roteia o tráfego externo para os *Services* apropriados.

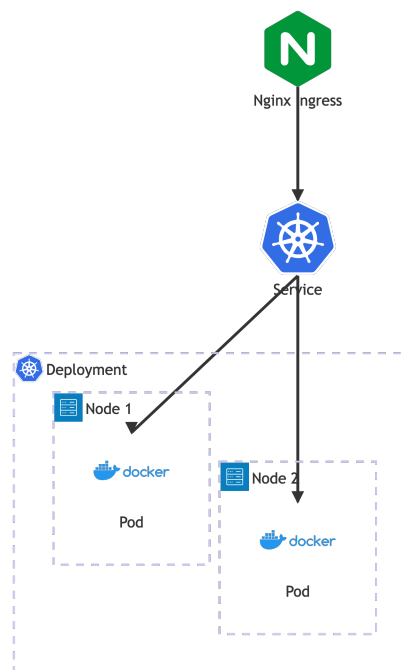


Figura 12 – Estrutura de deploy das aplicações

Fonte: Elaborado pelos autores.

A Figura 12 ilustra esta estrutura. O *Deployment* distribui os *Pods* entre diferentes nós do *cluster*, garantindo alta disponibilidade – se um nó falhar, os *Pods* nos demais nós continuam atendendo requisições. O *Service* abstrai a localização dos *Pods*, fornecendo um

endpoint estável para o Ingress Controller, que por sua vez expõe a aplicação externamente através das regras de roteamento configuradas.

5.1.1.3 Fluxo de Dados

O fluxo de dados no *got.it* é projetado para otimizar as operações de acordo com as características de cada camada arquitetural. Para as funcionalidades centrais de leitura e escrita de notas, o processo se inicia no *frontend* com a interação do usuário no editor Tiptap. As modificações no documento são gerenciadas pelo sistema de sincronização colaborativa do Tiptap, que emprega o algoritmo *Conflict-free Replicated Data Type* (CRDT) via Yjs, uma biblioteca JavaScript para desenvolvimento de aplicações colaborativas multiusuário, para resolver conflitos de edição simultânea. Posteriormente, as modificações são enviadas ao *backend* Convex por meio de uma *mutation*. Essa operação persiste as alterações de maneira transacional no PostgreSQL. O modelo reativo do Convex, por sua vez, distribui essas atualizações em tempo real para todos os clientes que estão subscritos ao documento através de uma *query*, utilizando *WebSockets* para garantir a consistência do estado entre os usuários.

Para interações que envolvem Inteligência Artificial, o fluxo de dados é mais complexo e atravessa todas as quatro camadas. Considerando o caso de uso do *autocomplete*, quando o usuário está digitando e pausa momentaneamente, o *frontend* identifica o contexto atual (parágrafo sendo editado) e envia uma requisição ao *backend* através de uma *action* do Convex. Esta *action*, executada no servidor, realiza uma chamada HTTP à API do LM Studio na camada de IA, enviando o contexto e solicitando uma sugestão de continuação. O modelo Qwen 3 4B processa a requisição e retorna a sugestão, que é encaminhada de volta através das camadas até o *frontend*, onde é apresentada ao usuário de forma não-obstrutiva. Todo este fluxo ocorre em menos de um segundo devido ao tamanho reduzido do modelo e às otimizações de inferência do LM Studio, garantindo uma experiência fluida de digitação assistida.

O fluxo de busca semântica exemplifica a integração completa de todas as tecnologias da plataforma. Quando o usuário realiza uma consulta em linguagem natural, o *frontend* envia a pergunta ao *backend* via uma *action*. O Convex então orquestra múltiplas operações: primeiro, invoca o modelo Nomic Embed para converter a consulta em um vetor de *embedding*; em seguida, executa uma busca de similaridade vetorial no PostgreSQL utilizando a extensão pgvector, que retorna os fragmentos de notas mais semanticamente relevantes; finalmente, constrói um *prompt* contendo a consulta original e os fragmentos recuperados, e envia para o modelo GPT-OSS 20B via LM Studio. O modelo processa o *prompt* expandido e gera uma resposta fundamentada no conteúdo real das notas do usuário. Esta resposta é então retornada através das camadas até ser exibida na interface. O sistema implementa *streaming* de resposta, permitindo que o *frontend* exiba progressivamente

o texto conforme ele é gerado pelo modelo, melhorando a percepção de responsividade mesmo para consultas que levam vários segundos para serem completamente processadas.

A comunicação entre as camadas é padronizada através de formatos e protocolos bem estabelecidos. O *frontend* e o *backend* Convex comunicam-se exclusivamente através da API gerada automaticamente pelo Convex, que utiliza *WebSockets* para comunicação bidirecional e HTTP para operações síncronas. O *backend* e a camada de IA comunicam-se via HTTP/REST seguindo o padrão de API da OpenAI, o que permite substituir ou complementar os modelos locais com serviços externos sem modificações significativas no código. A persistência de dados entre o Convex e o PostgreSQL é gerenciada internamente pela plataforma, abstraindo os detalhes do protocolo de conexão com o banco. Esta padronização de interfaces facilita testes, *debugging* e potenciais substituições futuras de componentes.

5.1.1.4 Decisões Arquiteturais Principais

A arquitetura do *got.it* reflete uma série de decisões técnicas tomadas ao longo do desenvolvimento, cada uma justificada por requisitos específicos do projeto e restrições de recursos. Esta subseção discute as decisões arquiteturais mais significativas e seus impactos no sistema.

Por serem utilizados apenas quatro nós no *cluster*, optou-se pelo k3s, uma distribuição leve do Kubernetes que reduz significativamente o *overhead* do plano de controle e simplifica instalação e *upgrades* por meio de um único binário. Além disso, ele permite escalonar contêineres em nós de controle, algo que o Kubernetes não faz por padrão. Em um cenário de recursos limitados, essa escolha permitiu reservar a maior parte de CPU e memória para as cargas da plataforma (Convex, NGINX Ingress, ArgoCD, CloudNativePG e serviços de aplicação) sem perder compatibilidade com o ecossistema Kubernetes e com os operadores necessários ao projeto.

A separação dos modelos de IA do *cluster* Kubernetes representa uma das decisões de engenharia mais importantes do projeto. Diferentemente de uma arquitetura onde todos os componentes executariam no *cluster* Kubernetes, optou-se por manter a infraestrutura de modelos em um servidor dedicado fora do *cluster*. Esta decisão foi motivada por diversos fatores técnicos e práticos. Primeiro, os modelos de linguagem de grande porte apresentam requisitos de *hardware* muito específicos, particularmente em termos de memória GPU (VRAM) e capacidade de processamento vetorial, que são substancialmente diferentes dos requisitos de componentes tradicionais de aplicações *web*. Destaca-se principalmente que apesar de algumas soluções como o *llmd*, o Kubernetes ainda não possui uma solução completa para executar a inferência de modelos de linguagem. Segundo, a separação permite que a infraestrutura de modelos seja escalada, atualizada ou substituída de forma completamente independente do restante do sistema. Durante o desenvolvimento,

esta separação se mostrou valiosa ao permitir experimentação com diferentes modelos, configurações de quantização e versões do LM Studio sem impactar a disponibilidade do resto da plataforma. Terceiro, do ponto de vista de custos e complexidade operacional, manter os modelos separados simplifica o gerenciamento de recursos no *cluster* Kubernetes, que pode focar em orquestrar componentes stateless e mais facilmente escaláveis. A comunicação via HTTP/REST entre o Convex e o LM Studio introduz uma pequena latência adicional, mas esta é mínima, comparada ao tempo de inferência dos próprios modelos.

A escolha do Convex como plataforma de *backend* merece atenção especial por representar a terceira iteração da arquitetura de *backend* do projeto, conforme mencionado na Subseção 2.5.1. Inicialmente, o sistema utilizava um servidor Node.js com Express, seguindo uma arquitetura mais tradicional de API REST. Esta primeira versão foi substituída por uma implementação em Ruby on Rails com o objetivo de aumentar a produtividade através das convenções e ferramentas do *framework*. No entanto, a implementação do editor colaborativo baseado em Tiptap revelou-se excepcionalmente complexa nesta arquitetura, particularmente a sincronização em tempo real do estado do documento entre múltiplos clientes, que é fundamental para a proposta de valor da plataforma. A migração final para o Convex resolveu este problema de forma elegante ao oferecer primitivas de comunicação em tempo real como funcionalidade nativa, abstraindo toda a complexidade de gerenciamento de *WebSockets*, sincronização de estado e resolução de conflitos. Adicionalmente, o modelo de programação do Convex, onde tanto o *frontend* quanto o *backend* são escritos em TypeScript com tipagem compartilhada, eliminou uma classe inteira de erros relacionados a incompatibilidades de interface entre camadas. A arquitetura *serverless* do Convex também simplificou consideravelmente o *deployment* e a operação do *backend*, já que a plataforma gerencia automaticamente aspectos como escalabilidade, balanceamento de carga (*load balancing*) e disponibilidade.

Todavia, um aspecto negativo desta escolha é a dificuldade de realizar operações arbitrárias, visto que o Convex demanda um ambiente de execução específico para suas *queries* e *mutations*. Isso gerou alguns problemas, inclusive pela dependência específica do *Node.JS 18*, uma versão relativamente antiga, apesar de ser designada como *LTS* (*Long Term Support*, versão com suporte de longo prazo).

5.1.2 Modelagem de Dados e Schema do Banco

O *got.it* utiliza o Convex como *backend-as-a-service*, que oferece um modelo de dados baseado em documentos com tipagem forte através de *schemas* declarativos. Esta seção apresenta a estrutura de dados da plataforma, detalhando as principais entidades do sistema e as decisões arquiteturais que fundamentaram o *design* do modelo de dados.

Diferentemente de bancos de dados relacionais tradicionais, o Convex adota uma

abordagem híbrida que combina a flexibilidade de documentos NoSQL com validação de tipos rigorosa em tempo de desenvolvimento. Os *schemas* são definidos em TypeScript utilizando a biblioteca de validação do Convex (`convex/values`), garantindo consistência e segurança de tipos em toda a aplicação.

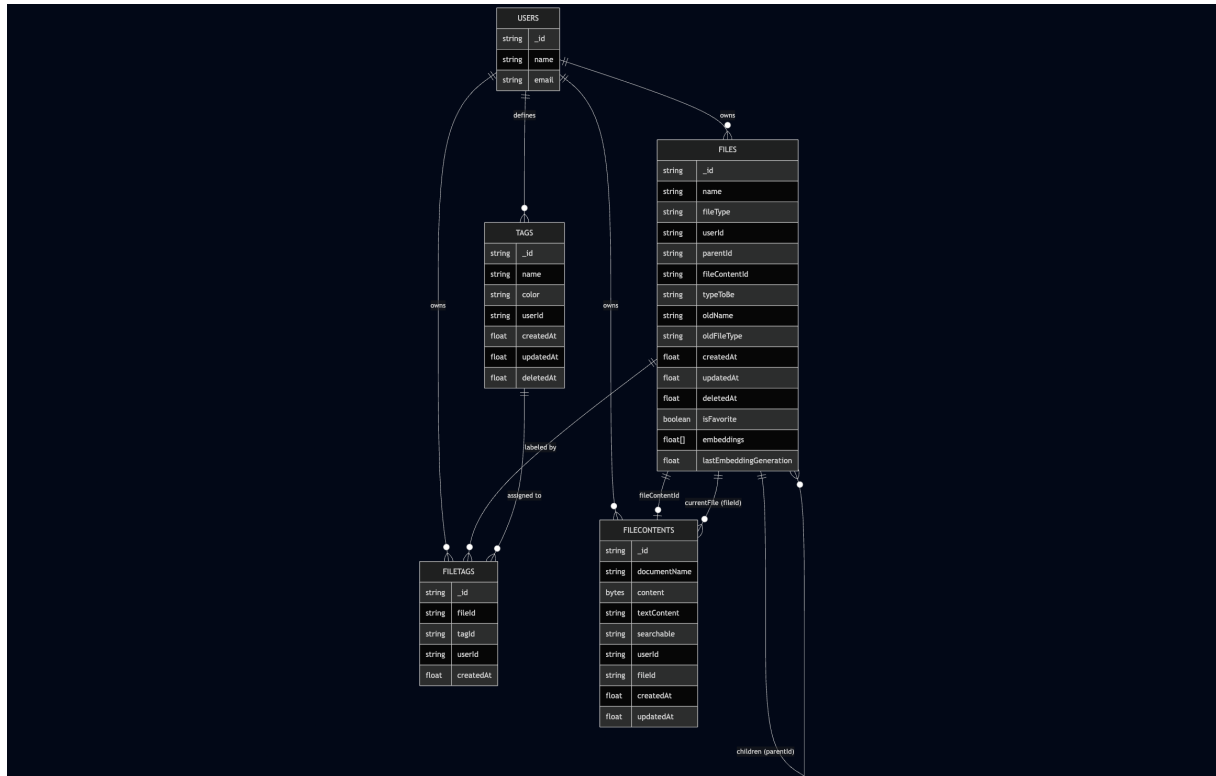


Figura 13 – Diagrama de entidade-relacionamento do *got.it*.

Fonte: Elaborado pelos autores.

A Figura 13 mostra um diagrama com as principais entidades e seus relacionamentos do *got.it*. É possível observar que o sistema é composto por quatro entidades principais: *Users*, *Files*, *Tags* e *FileContents*. A entidade *Files*, no entanto, possui uma modelagem mais complexa do que o diagrama sugere, pois representa diferentes tipos de arquivos e, portanto, merece uma análise mais detalhada.

O *design* do modelo de dados do *got.it* reflete diversas decisões arquiteturais com o objetivo de equilibrar *performance*, complexidade operacional e experiência do usuário. Esta seção detalha as principais escolhas de modelagem e suas justificativas.

5.1.2.1 Design Polimórfico com União Discriminada

A entidade **Files** implementa um sistema de arquivos hierárquico completo, suportando pastas, arquivos e estados transitórios para atualizações otimistas (*optimistic updates*) na interface do usuário. O *design* utiliza uma união discriminada para representar diferentes tipos de nós na hierarquia.

A Tabela 2 resume os cinco tipos de nós na hierarquia e seus campos específicos:

Tabela 2 – Tipos de arquivo e campos específicos

Tipo	Propósito	Campos Exclusivos
<code>root_folder</code>	Pasta raiz de cada usuário	Nenhum (apenas campos comuns)
<code>folder</code>	Pastas organizacionais	<code>parentId</code> , <code>isFavorite</code>
<code>file</code>	Notas/documentos finalizados	<code>parentId</code> , <code>fileContentId</code> , <code>isFavorite</code> , <code>embeddings</code> , <code>lastEmbeddingGeneration</code>
<code>draft_file</code>	Estado transitório (criação)	<code>parentId</code> , <code>typeToBe</code>
<code>renaming_file</code>	Estado transitório (renomeação)	<code>parentId</code> , <code>oldName</code> , <code>oldFileType</code> , <code>fileContentId</code> (opcional), <code>embeddings</code> (opcional)

Fonte: Elaborado pelos autores.

A decisão de utilizar uma união para representar diferentes tipos de arquivos em uma única tabela, ao invés de tabelas separadas para cada tipo, merece análise detalhada. Essa abordagem simplifica a consulta da hierarquia de arquivos, permitindo que operações de listagem e navegação sejam executadas com uma única *query*. Além disso, reforça a integridade da estrutura de árvore, ao centralizar as regras de relacionamento entre pais e filhos dos sistemas de notas.

No entanto, isso também traz alguns *trade-offs*, como a validação condicional mais complexa e o *schema* mais verboso. Além disso, a união de tipos complexa sobrecarrega o compilador do TypeScript, o que, associado ao sistema de tipos do Convex, prejudica o desempenho do *Language Server Protocol* (LSP).

5.1.2.2 Separação entre *Files* e *FileContents*

A entidade *File Contents* armazena o conteúdo real dos documentos, separado da estrutura de metadados em *Files*. Esta separação permite otimizar *queries* que necessitam apenas de metadados (nome, data de criação, hierarquia) sem carregar o conteúdo completo do documento.

Os campos *content* (formato binário) e *text content* (texto puro) coexistem para suportar diferentes necessidades: *content* armazena o documento completo em formato proprietário do editor (ProseMirror JSON), enquanto *textContent* mantém uma versão em texto puro para processamento por modelos de IA.

O campo *searchable* replica informações de *text content* especificamente para o índice de busca textual, demonstrando uma decisão de desnormalização controlada para otimização de *performance* em buscas.

A decisão de separar metadados (*files*) do conteúdo real dos documentos (*fileContents*) fundamenta-se em otimização de *performance* para operações comuns. Primeiro, garante-se que a listagem de arquivos carregue apenas metadados (nome, datas, ícones), sem o peso do conteúdo completo dos documentos. Isso torna o caso comum rápido, princípio importante da engenharia de *performance*. Além disso, como os metadados mudam com menos frequência que o conteúdo, isso permite estratégias de *cache* diferenciadas para os diferentes padrões de acesso aos dados.

O *trade-off* é a necessidade de um *join* (ou segunda *query*) ao abrir um documento, mas este custo é amortizado pela frequência muito maior de operações de navegação comparadas a abertura de documentos. Também são necessários ajustes para que a busca por conteúdo, e semântica funcionem sem a necessidade de ler ambas as tabelas.

5.1.2.3 Estados Transitórios e *Optimistic UI*

Os tipos `draft_file` e `renaming_file` implementam um padrão de *Optimistic UI*, onde a interface responde imediatamente a ações do usuário sem esperar confirmação do servidor.

Esta estratégia proporciona *perceived performance* superior, melhorando a experiência do usuário em sistemas colaborativos, ao custo de complexidade adicional no gerenciamento de estados transitórios.

5.1.2.4 Indexação Multi-modal para Busca

O *got.it* adota três modalidades de busca, cada uma baseada em uma estratégia distinta de indexação e otimização. A busca textual utiliza o mecanismo de *full-text search* do Convex aplicado ao conteúdo textual dos arquivos. Para isso, mantém-se um campo *searchable* que replica *textContent*, uma desnormalização proposital feita para obter maior desempenho nas consultas. Essa modalidade é ideal para buscas por palavras-chave, correspondências exatas ou trechos específicos, sempre respeitando o isolamento *multi-tenant* por meio do filtro de `userId`.

A busca semântica, por sua vez, opera sobre vetores de *embeddings* de 768 dimensões gerados pelo modelo *Nomic Embed Text*. Esses vetores são armazenados diretamente junto aos metadados dos arquivos, o que reduz a latência ao evitar *joins* ou consultas adicionais. Um campo auxiliar, *lastEmbeddingGeneration*, permite regenerar apenas os *embeddings* que ficaram desatualizados após modificações no conteúdo. Assim como nas demais modalidades, o filtro de *userId* garante que os resultados permaneçam confinados ao espaço do usuário.

Por fim, a busca por metadados oferece consultas rápidas baseadas em informações estruturadas dos arquivos, como tipo, data de criação, marcações e atributos similares.

Essa modalidade possibilita operações como listar arquivos favoritos, documentos recentes ou itens filtrados por características específicas, complementando as buscas textual e semântica de forma eficiente.

5.1.2.5 Armazenamento de *Embeddings*: Co-localização versus Tabela Separada

A decisão de armazenar vetores de *embeddings* diretamente na tabela *files*, em vez de mantê-los em uma tabela separada, representa um equilíbrio entre desempenho e tamanho dos documentos. A principal vantagem dessa co-localização é a redução de latência, já que a busca vetorial não exige *joins* ou múltiplas consultas. Além disso, todas as atualizações (tanto de metadados quanto dos próprios *embeddings*) tornam-se atômicas, sendo realizadas em uma única operação.

Por outro lado, essa escolha aumenta o tamanho de cada documento, já que vetores de, por exemplo, 768 *floats* adicionam cerca de 6 KB a cada registro. Isso significa que consultas que não dependem de *embeddings* acabam transferindo mais dados do que o necessário, ainda que projeções do Convex atenuem o problema.

Considerando a frequência esperada de operações de busca semântica no *got.it*, a co-localização foi considerada vantajosa, privilegiando latência em detrimento de tamanho de documentos.

5.1.2.6 Segurança *Multi-tenant* e Isolamento de Dados

Todos os índices do sistema incluem *userId* como campo de filtro, o que implementa uma camada de isolamento responsável por impedir qualquer vazamento de dados entre usuários. Além disso, o sistema adota uma estratégia de segurança em múltiplos níveis. As *queries* e *mutations* do Convex realizam validações explícitas de *userId*, garantindo que cada operação só retorne ou modifique dados pertencentes ao usuário autenticado. Essa verificação é reforçada por um *middleware* de autenticação, que bloqueia requisições sem credenciais válidas antes mesmo que alcancem a camada de dados. Mesmo os índices usados para busca textual ou vetorial são construídos de forma isolada, sempre filtrando resultados por *userId*, o que impede que respostas ultrapassem fronteiras entre usuários.

Essa combinação de mecanismos cria uma arquitetura de *defense in depth* (defesa em profundidade), assegurando que, mesmo que uma *query* específica contenha falhas, a segurança *multi-tenant* do sistema permaneça intacta.

5.1.2.7 *Soft Delete* para Recuperação de Dados

O campo opcional *deletedAt* implementa o padrão de *soft delete*, no qual os registros não são removidos fisicamente do banco de dados, mas apenas marcados como deletados. Essa abordagem se justifica por três fatores principais: ela permite que usuários recuperem

arquivos excluídos acidentalmente, preserva um histórico completo de operações para fins de auditoria e evita problemas de integridade referencial, já que documentos que dependem de um *parentId* continuam consistentes mesmo após a remoção lógica de um item.

O principal *trade-off* dessa decisão é a necessidade de filtrar `deletedAt === undefined` em todas as consultas que não devem exibir itens deletados, o que adiciona complexidade à lógica das *queries*. Para mitigar o acúmulo desnecessário de dados, uma estratégia complementar de *garbage collection* pode remover definitivamente os registros marcados como deletados há mais de 30 dias, equilibrando a capacidade de recuperação com a obrigação de atender solicitações de exclusão permanente feitas pelos usuários.

5.1.3 Implementação do Sistema de Inteligência Artificial

Esta subseção apresenta os aspectos técnicos da implementação do sistema de inteligência artificial do *got.it*, com foco na arquitetura dos modelos de linguagem utilizados e nas técnicas de adaptação que viabilizam sua aplicação no domínio de notas acadêmicas. Conforme estabelecido na Subseção 2.3, o sistema emprega dois modelos distintos, Qwen 3 4B para *autocomplete* e GPT-OSS 20B para busca inteligente, ambos baseados na arquitetura de Transformadores.

5.1.3.1 Treinamento e Adaptação do Modelo Qwen

O desenvolvimento da funcionalidade de *autocomplete* inteligente no *got.it* demandou a especialização do modelo Qwen 3 4B para o domínio específico completar frases de notas acadêmicas. Embora modelos de linguagem de grande porte pré-treinados possuam conhecimento amplo sobre diversos domínios, sua aplicação direta para geração de continuções de texto em contextos acadêmicos específicos frequentemente resulta em sugestões genéricas ou inadequadas ao estilo característico de anotações estudantis. O processo de *fine-tuning* descrito nesta subseção teve como objetivo adaptar o modelo não primariamente para expandir seu conhecimento factual, mas para ensinar a estrutura, o formato e o estilo apropriados de continuções em contextos de estudo, permitindo que o modelo gere sugestões que pareçam naturalmente escritas por um estudante.

A primeira etapa do processo consistiu na criação de um *dataset* sintético representativo de notas acadêmicas em diversas disciplinas universitárias. A geração sintética foi necessária devido à ausência de *datasets* públicos de anotações estudantis reais que pudessem ser utilizados para treinamento. O processo de geração foi estruturado em duas fases sequenciais: na primeira fase, utilizou-se um modelo de linguagem *open-source* para gerar uma lista diversificada de temas comuns que aparecem em anotações de aulas em diferentes áreas do conhecimento universitário, abrangendo desde ciências exatas (algoritmos de ordenação, derivadas parciais) até ciências biológicas (crise adrenal, ciclo de Krebs). Na segunda fase, para cada tema identificado, o mesmo modelo foi instruído a

criar exemplos realistas de anotações que um estudante poderia elaborar durante ou após uma aula sobre aquele tema, incluindo elementos característicos como definições resumidas, exemplos práticos, conexões entre conceitos e observações pessoais.

A validação do *dataset* sintético gerado focou predominantemente nos aspectos formais e estruturais das anotações, e não na correção factual do conteúdo apresentado. Esta escolha reflete a natureza do objetivo do *fine-tuning*: o processo visa ensinar ao modelo o *estilo* e a *forma* apropriados de continuções textuais em contextos acadêmicos, não expandir sua base de conhecimento factual, que já foi estabelecida durante o pré-treinamento. Durante a validação, verificou-se se as anotações sintéticas apresentavam estruturas típicas de notas estudantis (uso de tópicos, abreviações, ênfases), transições naturais entre ideias e extensão apropriada para contextos de estudo. Após a validação, o *dataset* foi dividido em conjuntos de treinamento e teste, reservando-se uma porção dos dados (10%) para avaliação posterior da qualidade do modelo ajustado.

A primeira tentativa de *fine-tuning* utilizou um modelo base com apenas 500 milhões de parâmetros, escolhido inicialmente por sua menor demanda de recursos computacionais e tempo de treinamento. A técnica LoRA foi aplicada, conforme descrito anteriormente na Subseção 2.4.3, e o treinamento completo levou aproximadamente 3 horas no *hardware* disponível.

No entanto, a avaliação do modelo resultante não foi satisfatória. Após tentar resolver o problema, foram levantadas duas limitações que inviabilizavam sua aplicação prática no sistema. Primeiro, o modelo de 500M parâmetros demonstrou capacidade de raciocínio insuficiente para gerar continuções coerentes e contextualmente apropriadas, frequentemente produzindo sugestões semanticamente desconexas do contexto precedente ou contendo inconsistências lógicas evidentes. A análise qualitativa das gerações indicou que o modelo não possuía representações internas suficientemente ricas para capturar as nuances do contexto da anotação e gerar continuções plausíveis, limitação esta inerente ao tamanho reduzido do modelo de base. Segundo, a estratégia inicial de *tokenização* do *dataset* empregava uma abordagem ingênua com sobreposição entre as janelas de contexto, onde o mesmo trecho de texto aparecia tanto como entrada (contexto) quanto como saída esperada (continuação) em diferentes exemplos de treinamento. Esta sobreposição gerou instabilidade durante o *fine-tuning*, manifestando-se em flutuações erráticas nas métricas de perda e dificuldade de convergência, uma vez que o modelo recebia sinais contraditórios sobre qual deveria ser a continuação apropriada de um dado contexto.

Para resolver as limitações identificadas, o projeto migrou para o modelo Qwen 3 4B, que com aproximadamente oito vezes mais parâmetros, demonstra capacidade significativamente superior de compreensão contextual e geração coerente. Simultaneamente, a estratégia de *tokenização* foi reformulada para eliminar completamente a sobreposição entre janelas, garantindo que cada segmento de texto aparecesse em exatamente um

exemplo de treinamento e que houvesse separação clara entre a porção usada como contexto e a porção usada como alvo de predição. Apesar disso significativamente reduzir a quantidade de exemplos de treinamento, esta correção estabilizou o processo de treinamento, permitindo convergência das métricas de perda. O modelo final teve os parâmetros gerados mesclados com o modelo base e foi integrado ao sistema através do LM Studio conforme descrito na Subseção 5.1.1, atendendo aos requisitos de latência inferior a um segundo necessários para a funcionalidade de *autocomplete* em tempo real.

5.1.4 Desenvolvimento do *Backend*

O desenvolvimento do *backend* do *got.it* utilizou a estrutura de primitivas do Convex (*queries*, *mutations* e *actions*) para organizar a lógica de negócio da aplicação. Esta seção apresenta como o código foi estruturado em módulos funcionais e os padrões de desenvolvimento adotados para garantir uma boa manutenção e escalabilidade do sistema.

5.1.4.1 Organização e Estrutura de Módulos

O código do *backend* está organizado no diretório **convex/**, seguindo uma estrutura modular baseada em domínios funcionais. Cada módulo agrupa as primitivas relacionadas a uma funcionalidade específica do sistema, facilitando a localização de código e manutenção evolutiva.

5.1.4.1.1 Organização Interna dos Módulos

Cada módulo funcional agrupa suas primitivas em arquivos dedicados ou, em casos mais complexos, em subdiretórios. Por exemplo, o módulo **files/** contém:

- *Queries* para listagem de arquivos por pasta, busca de metadados e navegação hierárquica.
- *Mutations* para criação, atualização, movimentação, renomeação e exclusão (*soft delete*) de arquivos.
- *Actions* para operações que envolvem efeitos colaterais, como notificações ou processamento assíncrono.

Esta coesão funcional garante que toda lógica relacionada a um domínio esteja localizada, facilitando raciocínio sobre o sistema e reduzindo a carga cognitiva durante desenvolvimento e depuração.

5.1.4.2 Padrões de Desenvolvimento e Fluxos

O Convex organiza a lógica do *backend* em três primitivas principais, sendo elas *queries*, *mutations* e *actions*, cada uma com responsabilidades bem definidas. As *queries* são operações exclusivamente de leitura, realizando filtros, agregações e formatações, mas sendo incapazes de modificar o estado. Por serem funcionalmente puras, beneficiam-se da reatividade automática e do *cache* interno do Convex, garantindo respostas rápidas e consistentes. As *mutations* são responsáveis por qualquer modificação no banco de dados e sempre executam em um contexto transacional com garantias ACID (*Atomicity*, *Consistency*, *Isolation*, *Durability*), o que assegura consistência mesmo em cenários com múltiplas operações concorrentes. Antes de qualquer alteração. Já as *actions* lidam com operações que envolvem efeitos colaterais não suportadas nativamente no Convex, como chamadas HTTP e processamento assíncrono de comandos. Elas não possuem garantias transacionais próprias, mas podem chamar *mutations* para persistir resultados de forma atômica quando necessário.

5.1.5 Desenvolvimento do *Frontend*

O desenvolvimento do *frontend* do *got.it* utilizou React 19 como biblioteca base, seguindo uma arquitetura de componentes funcionais e composição. A navegação entre páginas é gerenciada pelo TanStack Router com roteamento baseado em arquivos, enquanto a comunicação com o *backend* ocorre via integração reativa com Convex. Esta seção detalha as duas áreas centrais do desenvolvimento: o gerenciamento de estado em múltiplas camadas e a implementação do editor de texto rico.

5.1.5.1 Gerenciamento de Estado

Aplicações React de maior complexidade requerem diferentes tipos de estado com características distintas: estado de interface local, preferências persistentes do usuário e dados sincronizados com o servidor. Para atender a esses requisitos, a arquitetura do *got.it* adota uma estratégia em camadas, utilizando ferramentas especializadas para cada categoria de estado.

Para gerenciamento de estado local que requer composição granular, utiliza-se a biblioteca Jotai. Essa abordagem é aplicada principalmente no contexto de interação com o agente de IA, onde múltiplos átomos independentes gerenciam a seleção de texto, arquivos selecionados e estado da conversa. A composição atômica permite que apenas os componentes que dependem de um estado específico sejam re-renderizados, evitando atualizações desnecessárias na interface. Configurações do editor que devem persistir entre sessões são gerenciadas com Zustand, utilizando o *middleware* de persistência para armazenamento em `localStorage`. Esse mecanismo mantém preferências como tamanho

de fonte, espaçamento entre linhas, família tipográfica, modo Vim e exibição de contadores de palavras e caracteres.

Dados provenientes do *backend* são acessados através de *queries* reativas do Convex, que substituem o padrão tradicional de busca e armazenamento local. A integração com TanStack Query via `ConvexQueryClient` permite *cache* automático e revalidação de dados. *Hooks* customizados como `useCurrentUser` e `useNotes` encapsulam a lógica de acesso, abstraindo detalhes de implementação dos componentes consumidores. As *subscriptions* em tempo real garantem que alterações no banco de dados sejam refletidas automaticamente na interface.

5.1.5.2 Editor de Texto Rico

O editor de notas do *got.it* é construído sobre o Tiptap, um *framework* de edição baseado no ProseMirror. Conforme apresentado na Seção 2.5.2, o Tiptap oferece uma arquitetura extensível via *plugins* modulares, permitindo a composição de funcionalidades específicas para a aplicação. O editor utiliza um conjunto de extensões organizadas por categoria funcional:

- **Formatação de texto:** estilos básicos (negrito, itálico, sublinhado), cores de texto e destaque, sobrescrito e subscrito.
- **Estrutura de conteúdo:** listas ordenadas, não-ordenadas e de tarefas (*checklists*), além de tabelas com controles de manipulação.
- **Blocos de código:** *syntax highlighting* para múltiplas linguagens de programação via integração com Lowlight.
- **Fórmulas matemáticas:** renderização de expressões LaTeX utilizando KaTeX.
- **Colaboração:** sistema de comentários com suporte a *threads* de resposta e resolução.
- **Acessibilidade:** modo Vim opcional para usuários familiarizados com o editor.

A sincronização do conteúdo do editor com o *backend* é realizada através do *hook* `useTiptapSync`, que conecta o estado do documento ProseMirror às primitivas do Convex. Este mecanismo garante que alterações sejam persistidas de forma automática e que múltiplos usuários possam visualizar atualizações em tempo real. O sistema inclui tratamento de erros de sincronização e reconexão automática em caso de perda temporária de conectividade.

5.1.5.3 Integração com Backend

A comunicação entre *frontend* e *backend* segue um padrão consistente baseado em *hooks* customizados. Para operações de leitura, utiliza-se o *hook useQuery* do Convex, que estabelece *subscriptions* reativas e atualiza automaticamente os componentes quando os dados mudam. Para operações com efeitos colaterais, como o envio de mensagens ao agente de IA, emprega-se o *hook useAction*. Essa padronização mantém os componentes desacoplados dos detalhes de implementação do *backend*, facilitando testes e manutenção evolutiva do código.

5.2 Testes e Avaliação

A seção abaixo descreve os procedimentos experimentais para refinar e validar o projeto desenvolvido pela equipe. Foram realizadas análises para validação do *Product-market fit* para estudantes, da interface visual e experiência de usuário, das funcionalidades existentes e desejáveis, do mapeamento de requisitos e priorização de tarefas para entregas parciais e, finalmente, para verificar análises de uso do sistema em produção, tanto no lado do cliente quanto do servidor.

5.2.1 Validação dos Requisitos Mapeados no *Lean Inception*

A primeira fase de testes consistiu em entrevistas informais com colegas de sala e indivíduos alinhados aos perfis de *personas* descritos na Subseção 3.1.3, principalmente com estudantes de graduação. Essa etapa validou que as jornadas de usuário mapeadas no *Lean Inception* correspondiam aos fluxos reais de uso esperados. Os resultados informaram o design da interface inicial do MVP (*Minimum Viable Product*) com validação prévia dos usuários.

5.2.2 Testes de Interface Visual com o Maze

A segunda fase ocorreu paralelamente ao desenvolvimento da infraestrutura do projeto, com a alocação de servidores, *clusters*, mecanismos de autenticação e demais elementos, descritos na Subseção 2.6. Utilizou-se a ferramenta Maze para avaliar a intuitividade da interface, coletando métricas de interação e feedback qualitativo dos usuários.

Foram criados protótipos interativos no Figma, uma ferramenta para *design* de interface de aplicações, representando as principais telas e fluxos do MVP do sistema. Realizou-se uma pesquisa com 26 usuários não-identificados, compatíveis com o perfil-alvo do *got.it* (estudantes universitários), recrutados através de divulgação controlada em grupos e canais apropriados. O foco desta etapa foi validar a UI (*User Interface* ou

Interface de Usuário) e UX (*User Experience* ou Experiência de Usuário) nas telas do Figma, priorizando o *design* e a usabilidade prática antes do desenvolvimento em produção.

Os participantes realizaram tarefas cotidianas típicas do sistema:

- Criar um subtítulo H2 na tela de anotações;
- Aplicar formatação em negrito a uma seleção de texto;
- Duplicar uma nota existente.

A ferramenta Maze rastreou cliques com *heatmaps*, tempo gasto em cada tarefa e pontos de atrito na interface.



Figura 14 – Resultados do Maze para a tarefa de "Criar um subtítulo H2" com monitoramento de *misclicks* e tempo médio em cada tela

Fonte: Elaborado pelos autores

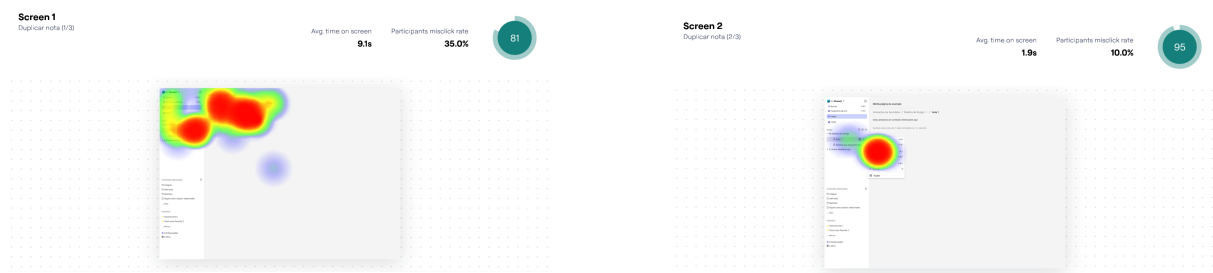


Figura 15 – Resultados do Maze para a tarefa de "Duplicar uma nota" com monitoramento de *misclicks* e tempo médio em cada tela

Fonte: Elaborado pelos autores

Os resultados demonstraram alta usabilidade: a criação de subtítulo H2 obteve 100% de sucesso (23 respostas) com NPS de 5.1/7.0, conforme visto na Figura 14; a formatação em negrito alcançou 100% de sucesso (21 respostas) com NPS de 6.4/7.0; e a duplicação de nota atingiu 100% de sucesso (20 respostas) com NPS de 6.1/7.0, conforme visto na Figura 15. Nenhuma das tarefas apresentou *drop-offs*.

Quanto à percepção geral, 20 usuários responderam à pergunta "*O que você achou da nossa plataforma?*", com análise de sentimentos indicando 80% de respostas positivas

e 20% neutras, sem respostas negativas. Perguntas de acompanhamento foram geradas automaticamente pela IA do Maze baseadas nas respostas iniciais.

As considerações dos usuários destacaram aspectos positivos: a plataforma foi percebida como intuitiva, com ícones e caminhos de clique simples e convenientes; as tarefas foram cumpridas facilmente em poucos segundos graças a botões intuitivos e palavras-chave; e o sistema foi considerado útil e esteticamente agradável, especialmente para manter anotações acadêmicas organizadas em um único local.

Esta etapa validou a UI e UX como aceitáveis, confirmando que o foco inicial do projeto em usabilidade prática foi bem-sucedido.

5.2.3 Pesquisa de Funcionalidades Desejadas com Google Forms

A terceira fase de testes consistiu em uma pesquisa qualitativa com usuários através de um formulário no Google Forms, visando mapear funcionalidades desejadas e validar prioridades de desenvolvimento. A pesquisa coletou 5 respostas de usuários alinhados ao público-alvo do *got.it*. O mapeamento revelou que todos os participantes possuíam idades entre 20 e 22 anos e estavam cursando graduação, com distribuição nos cursos de Engenharia de Produção, Medicina e Psicologia, e semestres variando entre o 4º e 10º.

Quanto aos objetivos ao utilizar o *got.it*, os usuários responderam (com múltiplas seleções permitidas):

- 80% buscava "Organizar melhor minhas tarefas e anotações";
- 80% desejava "Centralizar recursos e informações";
- 40% procurava "Tornar minha rotina de estudos mais prática";
- 40% buscava "Aumentar minha produtividade".

Notavelmente, nenhum participante selecionou "Monitorar e estabelecer hábitos saudáveis", indicando que funcionalidades de *habit tracking* não eram essenciais para o MVP neste contexto e poderiam ser implementadas em fases futuras.

Na avaliação do potencial de ajuda do *got.it*, com um intervalo possível de notas de 1 ("Não ajudaria") até 5 ("Ajudaria muito"), 20% atribuiu nota 4 ("Ajudaria") e 80% atribuiu nota 5 ("Ajudaria muito"), demonstrando alta confiança dos usuários na solução proposta. As solicitações de novas funcionalidades destacaram: lembretes e notificações para tarefas e anotações, tutoriais interativos da plataforma, integração com YouTube, exportação em HTML para compatibilidade com Anki, figurinhas customizáveis e funcionalidades de *chat* colaborativo.

Esta etapa resultou em ajustes significativos na priorização do desenvolvimento: funcionalidades não previamente mapeadas no *Lean Inception* foram incorporadas, as *sprints* foram re-priorizadas, novas *user stories* foram adicionadas e os *releases* foram concentrados em entregas de valor. As seguintes funcionalidades foram desenvolvidas baseadas neste *feedback*: lembretes de notas antigas, *embed* nativo de vídeos do YouTube, exportação de notas em HTML para Anki, e um novo painel de comentários e *chat* colaborativo por nota.

5.2.4 Análise de Métricas de Uso com PostHog e Grafana

A quarta fase de testes focou em análises de produção utilizando a ferramenta PostHog – uma plataforma de análise de produto que monitora métricas de engajamento e comportamento de usuários – e o Grafana, uma plataforma para monitorar métricas de infraestrutura e desempenho. Esta etapa permitiu validar o desempenho do sistema em ambiente real e informar decisões de desenvolvimento baseadas em dados de uso.

Foram analisadas as métricas de WAU (*Weekly Active Users* ou Usuários Semanais Ativos) e DAU (*Daily Active Users* ou Usuários Diários Ativos), conforme apresentado na Figura 16. A análise de picos revelou que aumentos significativos de atividade coincidiram com divulgação controlada nos grupos de WhatsApp da Engenharia Elétrica, com apoio do CEE (Centro de Engenharia Elétrica e de Computação), e com o lançamento de novas funcionalidades, como implementação de *tags*, comentários e *emojis*. Esses eventos demonstraram o impacto direto de divulgação e inovação contínua na adoção da plataforma.

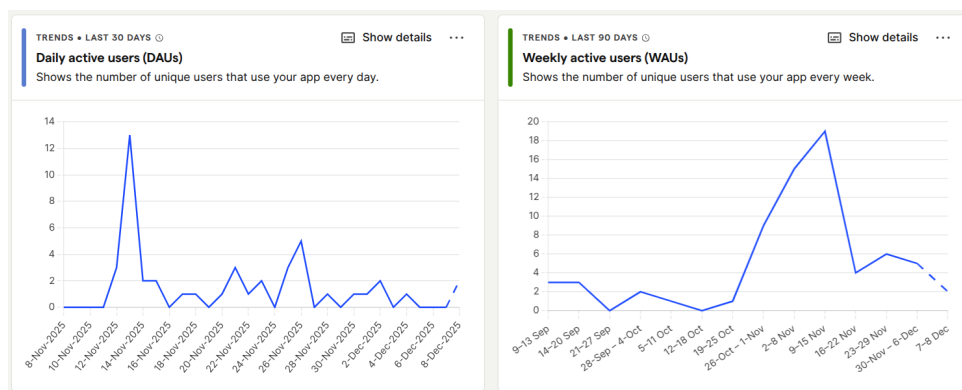


Figura 16 – Gráfico de WAU (*Weekly Active Users*) e DAU (*Daily Active Users*) no *got.it*

Fonte: Elaborado pelos autores.

Por outro lado, a análise dos vales em atividade identificou períodos associados a instabilidades temporárias na plataforma e épocas com menor divulgação ou retenção reduzida de usuários. Esses padrões sinalizaram a importância de esforços consistentes em *marketing*, divulgação estratégica e implementação de mecanismos de "gamificação" para manter o engajamento de longo prazo.

Adicionalmente, o PostHog disponibilizou pesquisas nativas de *feedback* diretamente na plataforma do *got.it*, sobre a experiência do usuário em produção. Estas observações informaram prioridades futuras de desenvolvimento e estratégias de retenção, como a adição de funcionalidades de desenho.

O monitoramento complementar com Grafana permitiu rastrear métricas de conteúdo gerado na plataforma. Até o momento da análise, em 8 de dezembro de 2025, foram registradas 73 notas criadas pelos usuários, totalizando 6773 palavras escritas. Esses dados validam o engajamento ativo dos usuários com a funcionalidade principal do sistema e demonstram uso consistente da plataforma para criação de conteúdo acadêmico.

5.2.5 Testes automatizados

Além dos testes-piloto com os usuários, foi incluído, no diretório `tests/` do repositório (RODRIGUES; NAKAMURA; SOUZA, 2025) um conjunto de testes elaborado com a ferramenta Playwright, com a finalidade de garantir a operação correta dos fluxos de funcionamento do projeto. O plano de teste, com os principais fluxos atualmente implementados, segue a estrutura geral descrita abaixo.

- Nome do fluxo (ex. Fluxo de autenticação), e a história do usuário correspondente, ou 'N/A' caso não aplicável.
- Casos de teste, incluindo dados ou comandos inválidos, bem como bifurcações na execução, como a criação de nota com um template específico.
- Pré-condições para execução dos testes, caso necessárias.
- Pós-condições a serem validadas.
- Riscos, como geração de objetos transientes no banco de dados a serem eliminados.
- Evidências de sucesso do teste.

6 Considerações Finais

Este capítulo apresenta as considerações finais sobre o desenvolvimento do *got.it*, uma plataforma *web* de aprendizado para estudantes. O sistema foi projetado para utilizar agentes de Inteligência Artificial de baixo custo, treinados para gerar conteúdo confiável a partir das anotações do usuário por meio de processos de RAG (*Retrieval-Augmented Generation*), visando agilizar o fluxo de estudo cotidiano.

A metodologia *Lean Inception* foi fundamental para estimar o tempo e os recursos necessários, bem como para definir o escopo do *Minimum Viable Product* (MVP). O planejamento detalhado das arquiteturas de *software*, infraestrutura e CI/CD permitiu maior agilidade nas entregas e resultou na finalização do produto dentro do prazo e com boa recepção pelo público-alvo.

Os objetivos do projeto foram alcançados com sucesso. O sistema incorpora funcionalidades essenciais de editores de texto rico, como destaques de texto e cor de fundo, ajuste de fonte, suporte à equações escritas em LaTeX, formatação de blocos de código, além de diversos outros recursos sugeridos pelos usuários durante a fase de testes. A plataforma permite a organização de notas em diretórios, suporte a imagens e exportação para formatos como TXT, PDF e HTML. Adicionalmente, o potencial das LLMs foi explorado na implementação de um *autocomplete* inteligente e de um *chatbot* de "Perguntas e Respostas", completando o conjunto de funcionalidades planejadas. Algumas dessas funcionalidades são ilustradas na Figura 17, com uma captura de tela do sistema *got.it* em produção.

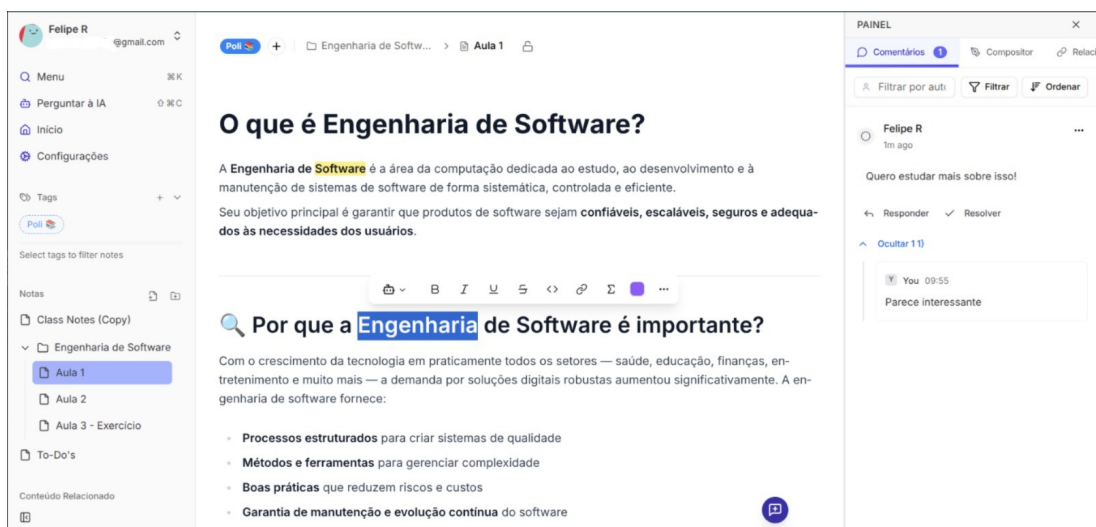


Figura 17 – Visão geral do produto final com destaque para o editor de notas.

Fonte: Elaborado pelos autores.

6.1 Resultados e Conclusões

O projeto atingiu resultados satisfatórios e cumpriu os objetivos propostos na Seção 1.1. A aplicação da metodologia *Lean Inception* permitiu um levantamento e priorização eficazes dos requisitos, resultando em um MVP que foi lançado em produção e validado por um público de usuários reais. O *feedback* coletado foi consolidado e documentado, orientando as etapas subseqüentes do desenvolvimento.

A pesquisa da literatura e a análise (*benchmark*) de soluções existentes no mercado levaram a discussões que refinaram a solução, com uma arquitetura flexível e robusta. O desenvolvimento seguiu as boas práticas da Engenharia de *Software*, garantindo a qualidade do produto final. Os requisitos levantados na fase inicial anteciparam com sucesso diversas demandas dos usuários, como a inserção de imagens e a exportação de notas para *flashcards*, que foram confirmadas durante os testes do MVP.

6.2 Contribuições

A principal contribuição deste trabalho reside na concepção e implementação da arquitetura do *got.it*, projetada com foco em escalabilidade, segurança (proteção contra ataques como DDoS), flexibilidade para suportar crescimento de usuários e novos recursos, além de desempenho e tolerância a falhas garantidos por soluções consolidadas. A arquitetura permite tanto hospedagem nos servidores da equipe quanto uma versão *self-hosted* para usuários que prefiram infraestrutura local.

O uso de modelos de linguagem *open-source* possibilitou *fine-tuning* otimizado para fins educacionais, reduzindo significativamente os custos operacionais e abrindo o projeto para contribuições da comunidade.

O projeto também oferece contribuições relevantes para a comunidade acadêmica e de desenvolvimento, incluindo uma arquitetura de referência *open-source* replicável que integra orquestração de microsserviços (Kubernetes, ArgoCD), banco de dados vetorial e aplicação reativa, com infraestrutura para hospedar aplicações complexas e LLMs em servidores *bare metal* com exposição segura via Cloudflare Zero Trust. Adicionalmente, o trabalho demonstra a democratização da IA na educação ao provar que funcionalidades avançadas, como busca semântica e RAG, podem ser oferecidas com modelos locais otimizados, sem dependência de APIs de terceiros. Por fim, a metodologia de Engenharia de Dados desenvolvida documenta o processo de criação de *datasets* sintéticos e técnicas de *Low-Rank Adaptation* (LoRA), servindo como guia para adaptar LLMs a domínios específicos com recursos computacionais limitados.

6.3 Perspectivas de Continuidade

Para aprimorar a usabilidade e expandir as funcionalidades do *got.it*, trabalhos futuros podem explorar os seguintes recursos:

- **Funcionalidade de desenhos:** Implementação de ferramentas para criação e edição de desenhos diretamente sobre as notas, em um *canvas* com *overlay* no texto. Sugere-se a utilização de bibliotecas de código aberto como Fabric.js.
- **Aplicativo móvel:** Desenvolvimento de versões para Android e iOS, com funcionalidades *offline*.
- **Verificador ortográfico:** Correção automática e sugestões de ortografia e gramática.
- **Acessibilidade:** Implementação de mais recursos para usuários com diferentes necessidades visuais, por exemplo.
- **Suporte a múltiplos idiomas:** Aumentar o número de traduções disponíveis para ampliar o alcance do sistema globalmente. Hoje, o *got.it* suporta apenas inglês e português.
- **Funcionalidade de voz:** Permitir entrada e saída de voz para ditado e leitura de notas.
- **Análise de imagens:** Uso de visão computacional – usando *Optical Character Recognition* (OCR), por exemplo – para automatizar a descrição de diagramas e imagens.
- **Integração com aplicativos externos de produtividade:** Compatibilidade com *softwares* de planilhas (como Microsoft Excel e Google Sheets), e também com calendários digitais, como o Google Calendar e o Outlook.
- **Bibliografia de referências:** Ferramenta para gerenciamento e citação de referências bibliográficas, similar ao *software* Zotero, por exemplo.
- **Análise de desempenho do usuário e "gamificação":** Incorporação de elementos de *gamificação* para aumentar o engajamento e monitorar o progresso do aprendizado. Criação de jornadas personalizadas com base no desempenho do usuário, e conexão com outras pessoas.
- **Gráficos e tabelas interativas:** Ferramentas para criação e manipulação de gráficos e tabelas diretamente nas notas.
- **Gerenciador de *shortcuts* de teclado:** Personalização de atalhos para melhorar a eficiência do usuário.

- **Sugestão de priorização de tópicos com base em sua complexidade:** Análise automática de conteúdo para sugerir ordem de estudo.
- **Criação de um *Marketplace* para extensões personalizadas:** Permitir que desenvolvedores criem e compartilhem extensões para o *got.it*, ampliando suas funcionalidades em uma "loja" oficial.

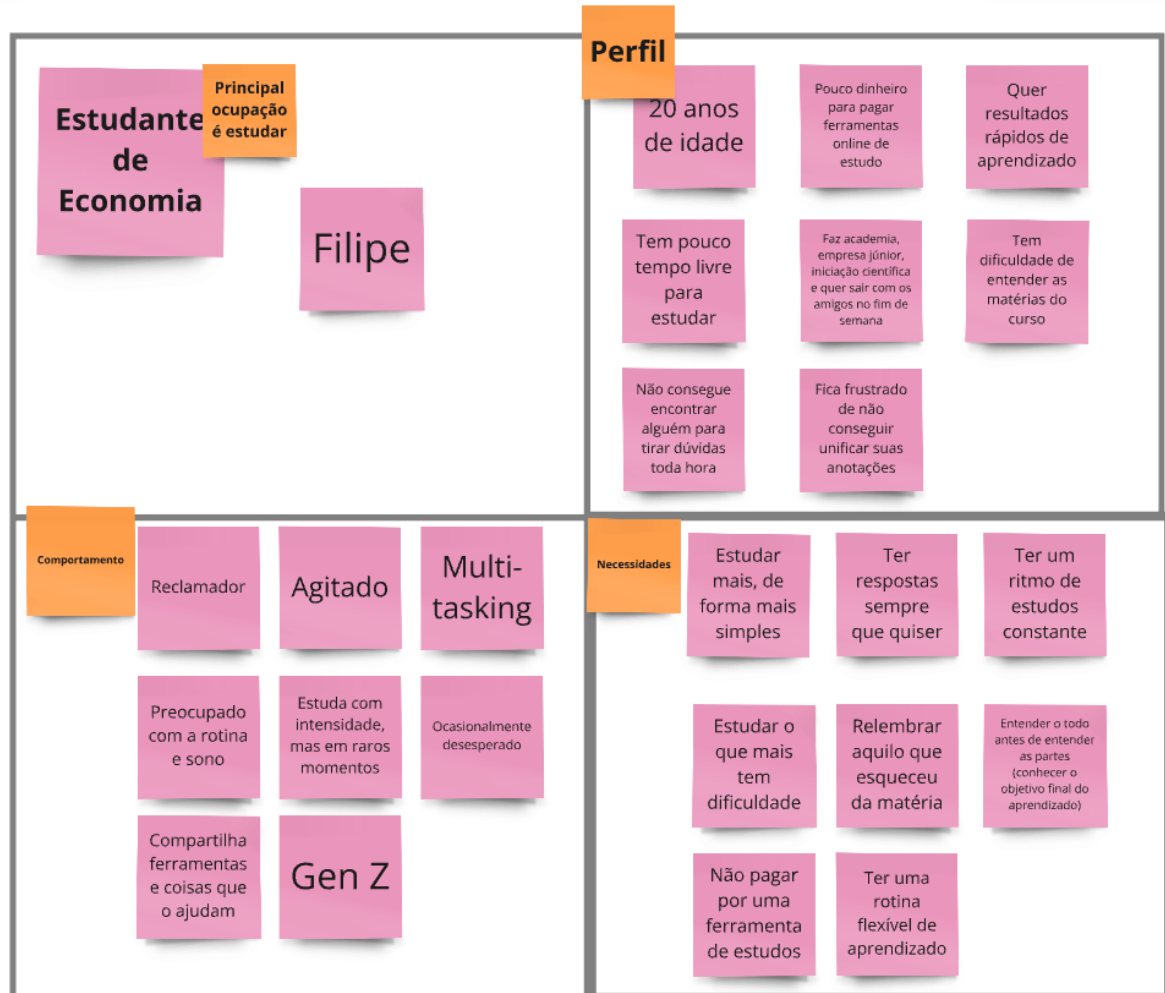
Referências

- CAROLI, P. *Lean Inception: como alinhar pessoas e construir o produto certo*. São Paulo, SP, BR: Editora Caroli, 2018. Citado 3 vezes nas páginas 16, 18 e 31.
- HONG, K.; TROYNIKOV, A.; HUBER, J. *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. 2024. <<https://research.trychroma.com/context-rot>>. Chroma Research. Acessado em: 17 de março de 2025. Citado na página 20.
- HOULSBY, N. et al. Parameter-efficient transfer learning for nlp. In: *ICML*. [S.l.: s.n.], 2019. Citado na página 22.
- HU, E. J. et al. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021. Citado na página 22.
- OPENAI. *Function Calling in the OpenAI API*. 2025. OpenAI Help Center. Acesso em: 1 nov. 2025. Citado na página 21.
- RODRIGUES, F.; NAKAMURA, M.; SOUZA, T. *got.it project repository*. 2025. <<https://github.com/thife455/gotit-proj>>. GitHub repository. Citado na página 63.
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. [S.l.]: Pearson, 2021. Citado na página 21.
- SUTHERLAND, J. *Scrum : a arte de fazer o dobro do trabalho na metade do tempo*. São Paulo, SP, BR: Leya, 2016. Citado na página 18.
- SUTTON, R. S.; BARTO, A. G. *Reinforcement Learning: An Introduction*. 2. ed. [S.l.]: MIT Press, 2018. Citado na página 21.
- TURING, A. M. Computing machinery and intelligence. *Mind*, Oxford University Press, v. 59, n. 236, p. 433–460, 1950. Citado na página 19.
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems*. Long Beach, CA, USA: NeurIPS, 2017. Citado na página 19.
- WEI, J. et al. Finetuned language models are zero-shot learners. In: *ICLR*. [S.l.: s.n.], 2022. Citado na página 23.
- YAO, S. et al. React: Synergizing reasoning and acting in language models. In: *Proceedings of ICLR*. [S.l.: s.n.], 2023. Citado na página 21.

Apêndices

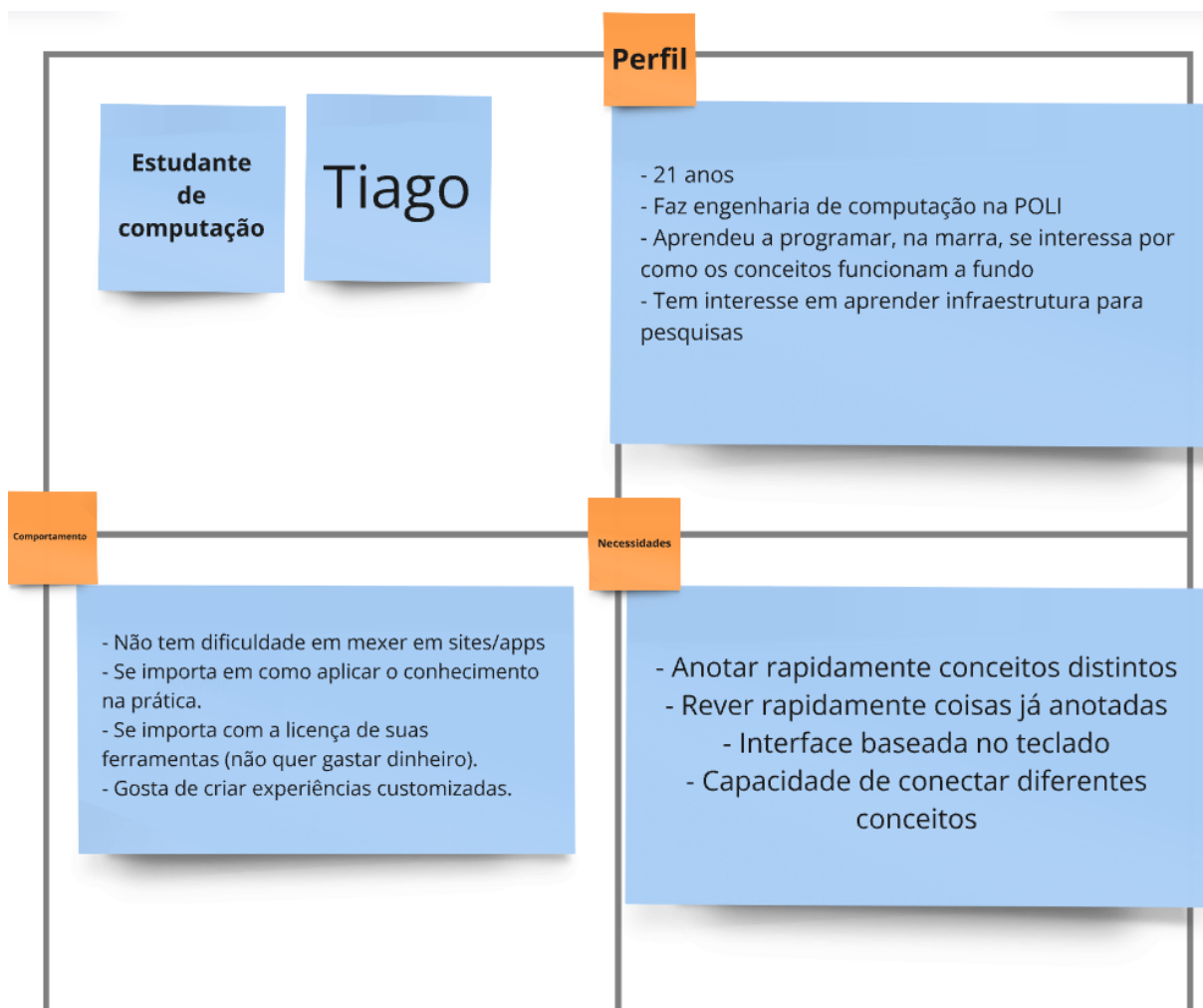
APÊNDICE A – Descrições das personas

Figura 18 – Descrição da persona do estudante de economia.



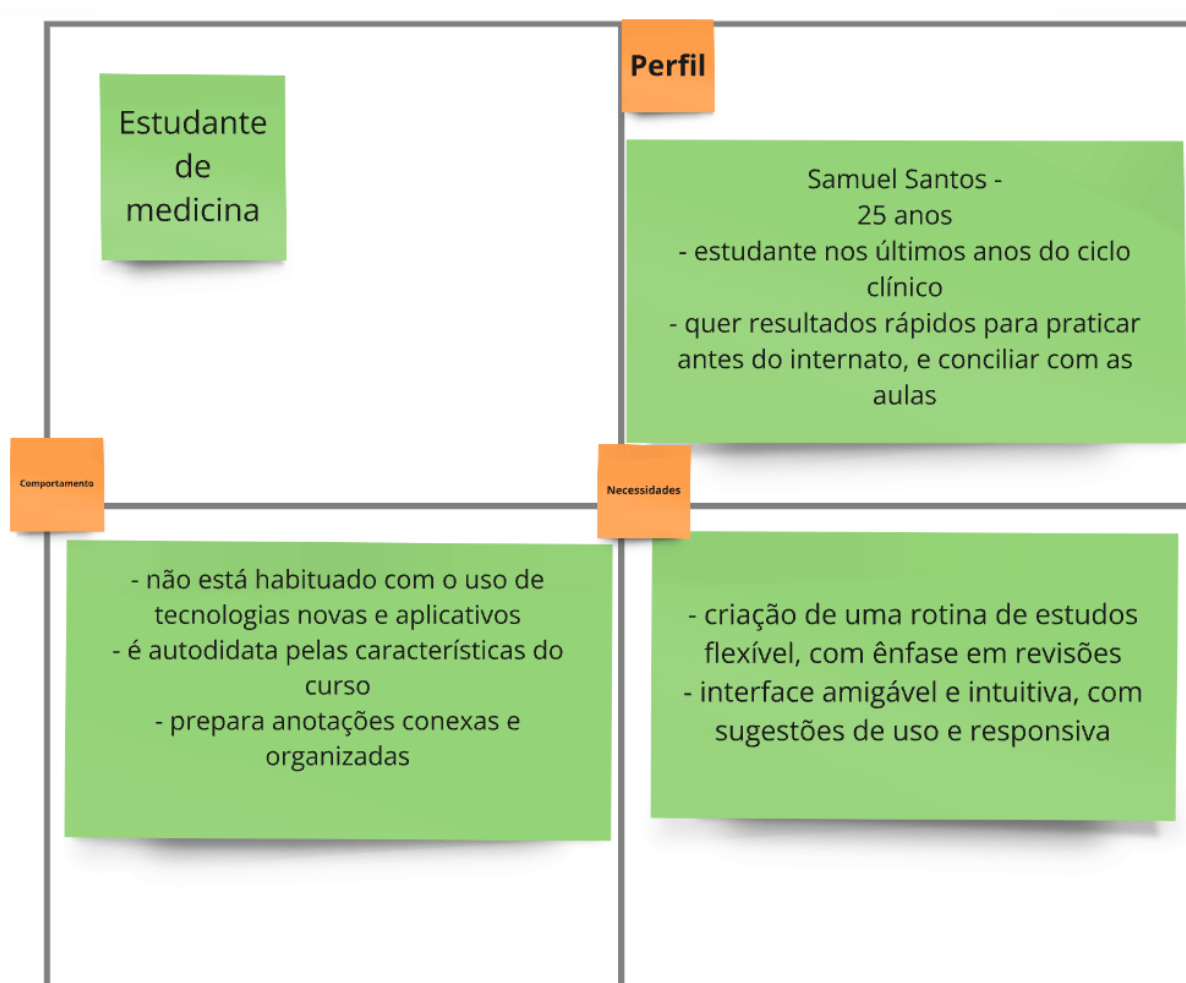
Fonte: Elaborado pelos autores.

Figura 19 – Descrição da persona do estudante de computação.



Fonte: Elaborado pelos autores.

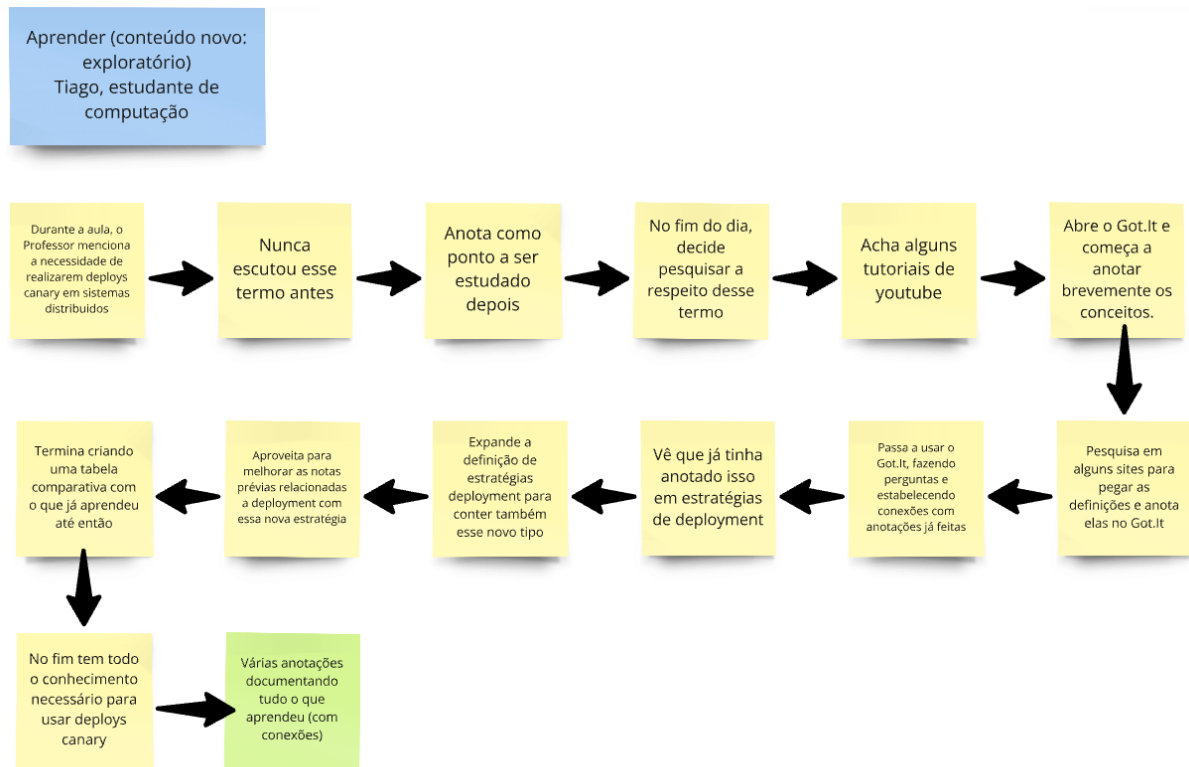
Figura 20 – Descrição da persona do estudante de medicina.



Fonte: Elaborado pelos autores.

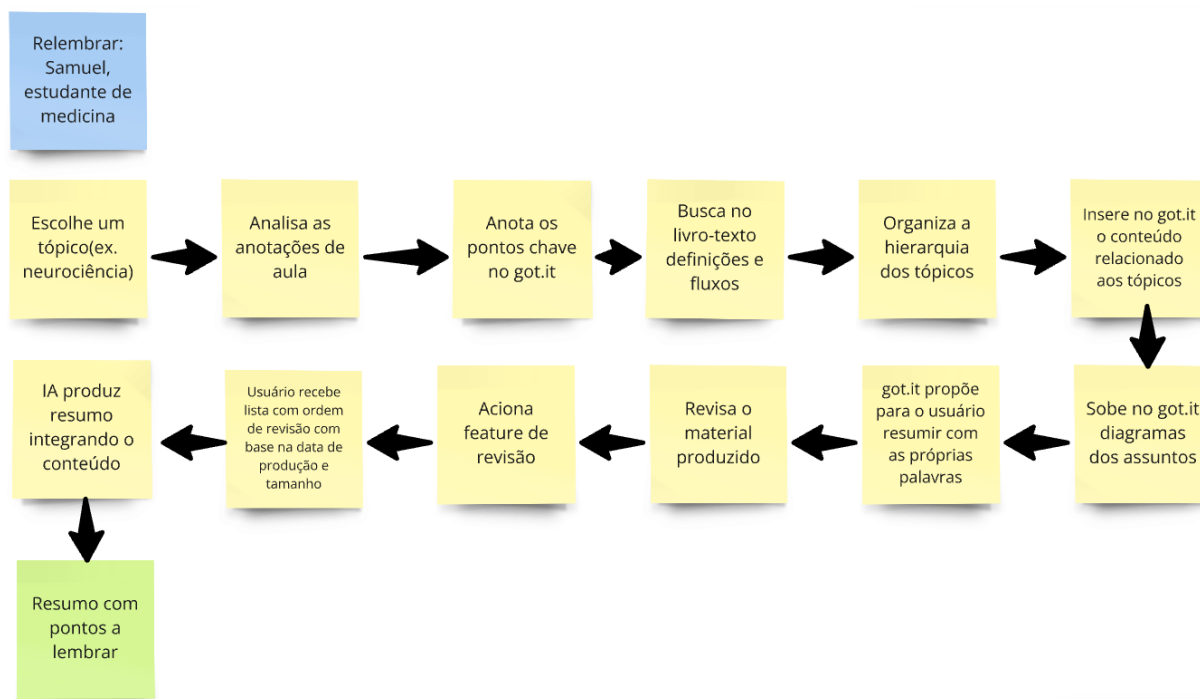
APÊNDICE B – Jornadas do usuário

Figura 21 – Descrição da jornada do processo de aprendizado exploratório.



Fonte: Elaborado pelos autores.

Figura 22 – Descrição da jornada de revisão.



Fonte: Elaborado pelos autores.